

© Brendan Choi 2021

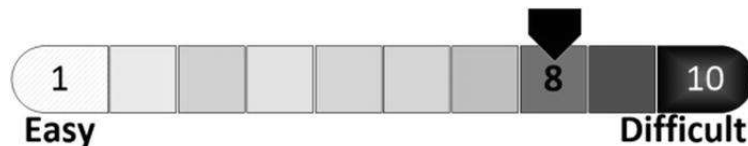
B. Choi, *Introduction to Python Network Automation*https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_15

15. Python Network Automation Labs: cron and SNMPv3

Brendan Choi¹

(1) Sydney, NSW, Australia

Eventually, you want to learn Python so you can write code to automate mundane tasks at work. After writing your code and developing your scripted application, you will want to run the scripts without you being in attendance. There is the Windows Task Scheduler on Windows systems, and on Linux, you have `cron` to help you schedule your script to run at 2 a.m. or to run periodically until you tell your Linux system to stop. You can write and schedule scripts to run automatically once you get comfortable with Python and Linux. You will also want to explore more about how Python works with SNMP so your company can consider SNMP monitoring using an in-house script; you may want to find out how Python interacts with SNMP in general. By the end of this chapter, you will be able to clone the GNS3 project, use `cron` to schedule tasks on Linux, and interact with routers and switches using SNMPv3. Along the way, we will borrow a Python community member's publicly shared code to explore SNMPv3 with Python and learn what it can do for us. This chapter aims to give you more exposure to different scenarios while practically applying Python to network automation scenarios.



Cron and SNMPv3 Labs

You have done well following the first 14 chapters of this book. You can write simple and functioning Python scripts for specific networking tasks. You will quickly realize that you need to find a way to run your scripted applications without actual user interaction or user intervention, possibly using a task scheduler. You probably realize that you might be able to tap into the power of Python automation and apply it to more traditional network concepts such as SNMP monitoring; the possibilities are endless. First, in preparation for the next chapter, you will learn how to make a real clone of the GNS3 project.

Cloning a GNS3 Project for the Next Lab

To prepare for our lab, you will make a true clone of the GNS3 project from Chapter 14. The GNS3 project export cloning method helped us to make a new project, but it compromised the original lab's configuration files, so in this sense, it is not a true clone of a GNS3 project. Sometimes this may not be the result you want for your labs. You might want to keep the original files intact and create a true clone of an old GNS3 project. With project export method of cloning, the original lab becomes unusable as the Dynamips files also get exported as the new project is created. A better way to create a new project without compromising the original project is to make a real duplicate of the original project folder and then reimport it with another name. You want to keep the `cmllab-basic` project as a fully working project, but at the same time, you do not want to create a new GNS3 project from the ground up. Let's learn how we can achieve this.

Task

- If you are still using the `cmllab-basic` project, first save all the Cisco devices' running configurations using the `copy running-config startup-config` command . Once the configurations of three routers and two switches have been saved, power off all devices normally.

Once all routers and switches are powered off, exit GNS3 so both GNS3 and GNS3 VM are closed completely. See Figure 15-1.

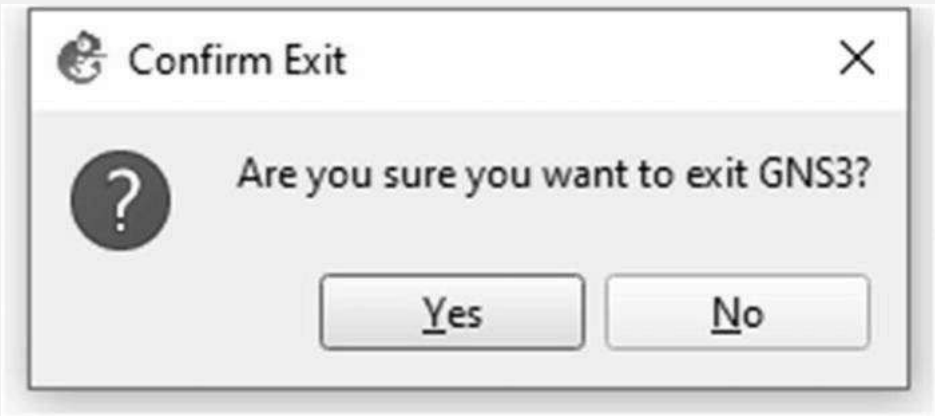


Figure 15-1. cmllab-basic GNS3 project, exiting GNS3 and GNS3 VM

Go to the `C:\Users\brendan\GNS3\projects` folder and make a copy of the `cmllab-basic` folder so it looks like Figure 15-2.

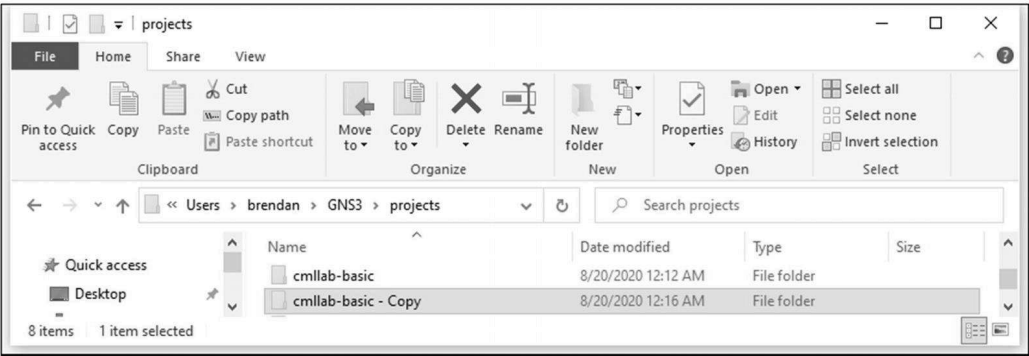


Figure 15-2. GNS3 project, making a copy of an existing project folder

Task

Go to the desktop of your Windows host PC and double-click the GNS3 icon on the desktop . Wait until both GNS3 and GNS3 VM become fully operational. See Figure [15-3](#).

4



Figure 15-3. GNS3 start desktop icon

When the project window opens, select “Open a project from disk,” open the `C:\Users\brendan\GNS3\projects\cmllab-basic - Copy` folder, select the `cmllab-basic` GNS3 project file , and then click the Open button. See Figure [15-4](#).

5

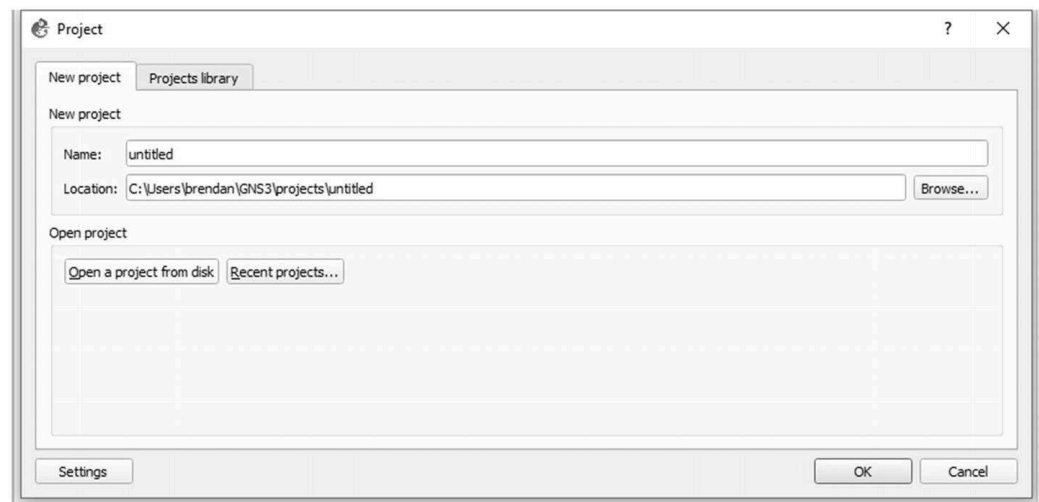


Figure 15-4. GNS3, selecting “Open a project from disk ”

Once the copy of the `cmllab-basic` project is fully launched, wait for the lab to open correctly and then select GNS3’s File ► Save project as . See Figure [15-5](#).

Task

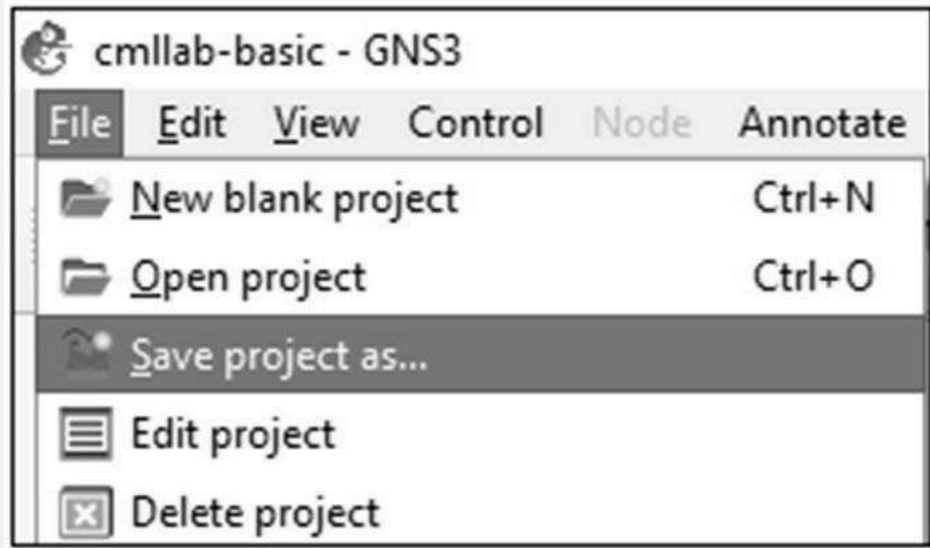


Figure 15-5. GNS3, "Save project as" menu item

Name your project and save the new project as `cmllab-devops`. See Figure 15-6.

7



Figure 15-6. GNS3, saving as a new project

- 8 Now, go back to the `C:\Users\brendan\GNS3\projects` folder, and you will find a new project folder called `cmllab-devops`. The temporary folder has served its purpose and has now become redundant; you can go ahead and delete the `cmllab-basic - Copy` folder permanently. See Figure 15-7.

Task

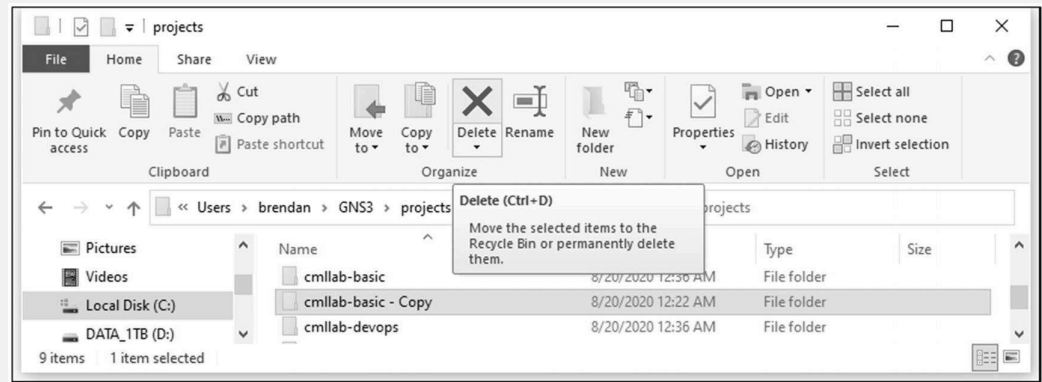


Figure 15-7. GNS3, deleting the redundant copy

Open the `cmllab-devops` folder, and you will find your new GNS3 project file with the correct name. Now power up all devices and check their running configurations. You have successfully made a true clone of the last GNS3 project. See Figure 15-8.

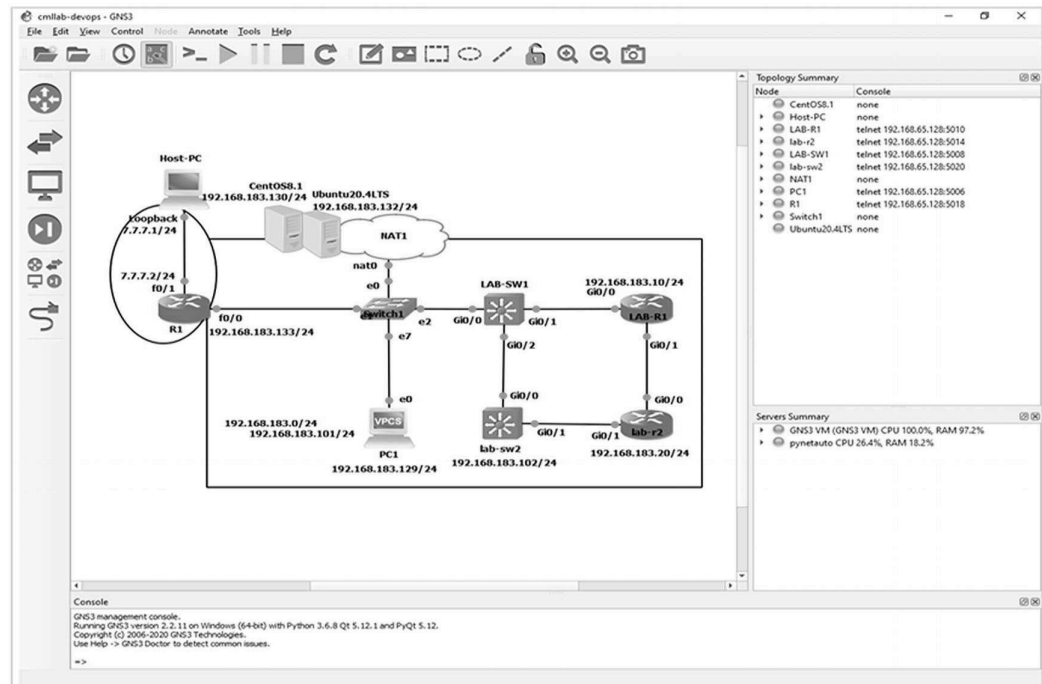


Figure 15-8. GNS3, fully cloned `cmllab-devops` project

- 10 To prepare you for the next part of our exploration, you will now add two more switches, as shown in Figure 15-9. First, delete `PC1` (VPCS) and replace it with `Lab-sw3` with a random IP address of `192.168.183.153/24` and with `Lab-sw4` with a random IP address `192.168.183.244/24`. Power on new switches and let the CPU and memory settle down. Also shown is a Cisco CSR 1000v router to be installed on VMware for a later IOS upgrade lab; this icon is only a placeholder, so you do not have to worry about the `csr1000v` router until the next chapter.

Task

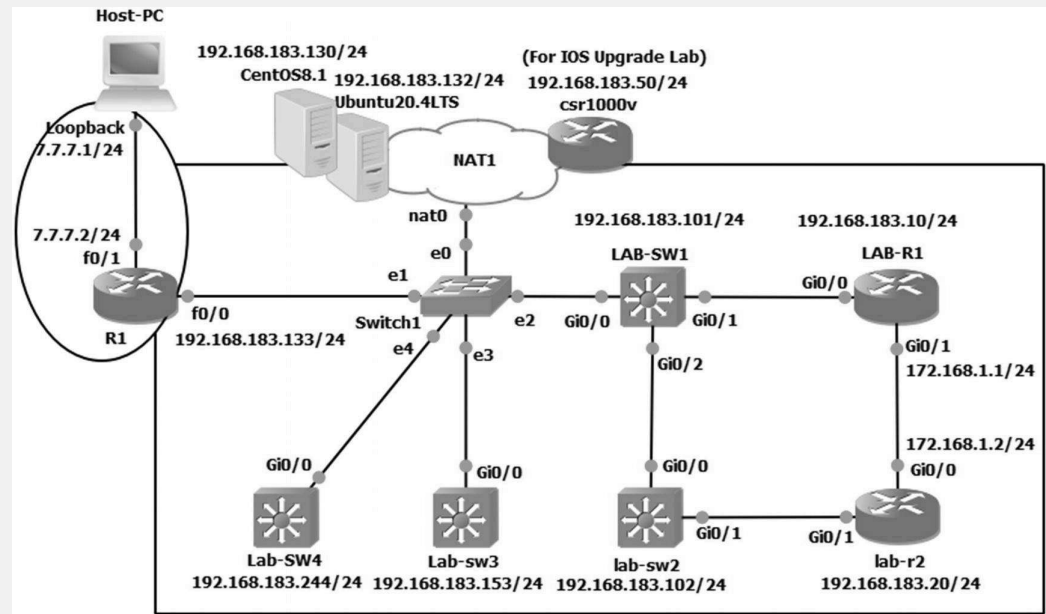


Figure 15-9. GNS3, adding two new switches

While you are configuring the new switches, run an extended ping from LAB-R1 and lab-r2: `ping ip 192.168.183.153 repeat counter 10000` and `"ping ip 192.168.183.244 repeat counter 10000`. Let the ping run continuously in the background.

LAB-R1#`ping ip 192.168.183.153 repeat 10000`

Type escape sequence to abort.

- 11 Sending 10000, 100-byte ICMP Echos to 192.168.183.153, timeout is 2 seconds:

.....

lab-r2#`ping ip 192.168.183.244 repeat 10000`

Type escape sequence to abort.

Sending 10000, 100-byte ICMP Echos to 192.168.183.244, timeout is 2 seconds:

.....

- 12 Once the switches are powered on, configure both switches using the following configurations. Make sure you disable IP routing on both switches and configure the gateway of the last resort. For LAB-sw3, the outbound traffic will go through LAB-R1, and for Lab-SW4, the unknown traffic will go through R1 first.

Lab-sw3 configuration:

Switch>`en`

Switch#`configure terminal`

Switch(config)#`hostname Lab-sw3`

Lab-sw3 (config)#`no ip routing`

Lab-sw3(config)#`ip default-gateway 192.168.183.10` !Route unknown destination to LAB-R1

Lab-sw3 (config)#`enable password cisco123`

Lab-sw3 (config)#`username pynetauto privilege 15 secret cisco123`

Lab-sw3 (config)#`line vty 0 15`

Lab-sw3 (config-line)#`login local`

Lab-sw3 (config-line)#`transport input all`

Lab-sw3 (config-line)#`exit`

```
# Task

Lab-sw3(config)#ip domain name pynetauto. local
Lab-sw3(config)#crypto key generate rsa !use 1024 bits
Lab-sw3(config)#interface vlan 1
Lab-sw3(config-if)#description "Native interface" !For later SNMP lab
Lab-sw3(config-if)#ip address 192.168.183.153 255.255.255.0
Lab-sw3(config-if)#no shut
Lab-sw3(config-if)#interface GigabitEthernet0/0
Lab-sw3(config-line)#description "1GB Main Connection" !For later SNMP lab
Lab-sw3(config-if)#end
Lab-sw3#copy running-config startup-config

Lab-SW4 configuration :
Switch>en
Switch#configure terminal
Switch(config)#hostname Lab-SW4
Lab-SW4(config)#no ip routing
Lab-SW4(config)#ip default-gateway 192.168.183.133 !Route unknown destination to R1
Lab-SW4(config)#enable password cisco123
Lab-SW4(config)#username pynetauto privilege 15 secret cisco123
Lab-SW4(config)#line vty 0 15
Lab-SW4(config-line)#login local
Lab-SW4(config-line)#transport input all
Lab-SW4(config-line)#exit
Lab-SW4(config)#ip domain name pynetauto. local
Lab-SW4(config)#crypto key generate rsa !use 1024 bits
Lab-SW4(config)#interface vlan 1
Lab-SW4(config-if)#ip address 192.168.183.244 255.255.255.0
Lab-SW4(config-if)#no shut
Lab-SW4(config-if)#end
Lab-SW4# copy running-config startup-config
```

After the configuration has been completed, you will get the new switches' ping responses . To stop and escape the continuous (extended) ping (or traceroute) request, use the Ctrl+^ or Ctrl+Shift+6 key.

```
LAB-R1#ping ip 192.168.183.153 repeat 10000
Type escape sequence to abort.
Sending 10000, 100-byte ICMP Echos to 192.168.183.153, timeout is 2 seconds:
.....
.....!!!!!!!!!!!!!!!!!!!!!!
13 Success rate is 85 percent (102/115), round-trip min/avg/max = 17/25/55 ms
lab-rl#ping ip 192.168.183.244 repeat 10000
Type escape sequence to abort .
Sending 10000, 100-byte ICMP Echos to 192.168.183.244, timeout is 2 seconds:
.....
.... !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Success rate is 88 percent (102/115), round-trip min/avg/max = 12/21/44 ms
```

Task



Remember the Ctrl+Shift+6 keys. If your router or switch hangs or you want to interrupt an operation, you can use this key combination to exit from the operation.

- 14 To confirm that all unknown outbound destinations from the new switches are routing through LAB-R1 and R1 routers, respectively, try to use `clear arp` and `traceroute` on Google's public DNS, 8.8.8.8 or 8.8.4.4.



If you have issues pinging the Google DNS, there might be an issue with your Internet's DNS or you have not configured your devices correctly. Another possibility is Windows' latest update causing GNS3 to malfunction and breaking the loopback interface used by GNS3. Even if you cannot ping the DNS IP address, leave it for now and move onto the next section.

```
Lab-sw3#clear arp
Lab-sw3#traceroute 8.8.8.8
Type escape sequence to abort.
Tracing the route to 8.8.8.8
VRF info: (vrf in name/id, vrf out name/id)
  1 192.168.183.10 36 msec 22 msec 16 msec
  2 192.168.183.2 43 msec * *
  3 * * *
[omitted for brevity]
30 * * *
Lab-SW4#clear arp
Lab-SW4#traceroute 8.8.4.4
Type escape sequence to abort.
Tracing the route to 8.8.4.4
VRF info: (vrf in name/id, vrf out name/id)
  1 192.168.183.133 1020 msec 41 msec 6 msec
  2 192.168.183.2 38 msec * *
  3 * * *
[omitted for brevity]
30 * * *
```

If you have completed the previous tasks, then you are ready to do more labs. At the end of the previous task, you should have two more IOSvL2 switches in your topology. Let's log into your Linux servers and master how to use Linux task scheduler, `cron`.

Quick-Start to the Linux Scheduler, cron

Earlier in this book, you were introduced to some Linux basics, and you also got familiar with building a Linux server to function as a server for multiple IP services. As you discover Linux, you will quickly realize that many of the things you could not or were not allowed to do on Windows

operating systems suddenly become possible with Linux systems. It allows you to install features and services quickly, and your IP services can be up and running in no time. If you are a Windows person, you will remember the good old Windows Task Scheduler; as the name suggests, Windows also has a task scheduler tool.

Linux has the equivalent task scheduler, but it is called `cron` (without a GUI). `cron` is the equivalent of the Windows Task Scheduler, but its operation is much simpler, reliable, and incredibly agile as it carries no burden of a GUI. Linux's `cron` tool is Linux's default tool for task scheduling. It can perform Linux's native command executions and set applications such as Python scripts to run at set intervals. Skilled Linux system administrators do a lot of work using automated shell-based scripting or other programming languages when working with system monitoring and backup tasks. To help with this task, the most commonly used tool in Linux is the `cron` tool, which serves the same purpose as Windows Task Scheduler for Windows systems. As you finish writing Python applications (scripts), you have to learn how to schedule your scripts to run at specific times and in a specific cycle for repetitive tasks, so it is essential that you master how to use `cron` and correctly run the scheduled tasks. Of course, if the company you work for has a bottomless IT budget, there is always an easy way out by purchasing an out-of-the-box task scheduler, but when they have hired smart people like you, why should your company pay for these free and open source tools?

In this section, you will be learning how to schedule tasks using `cron` in both Ubuntu and CentOS systems. The basic syntax is almost identical, but there are enough variations to treat them differently.

Ubuntu 20.04 LTS Task Scheduler: crontab

On a Ubuntu 20.04 server, `cron` is called `crontab`. You will write a greeting Python application and use it to learn task scheduling with Ubuntu's `crontab`. You will be required to follow the tasks on your Ubuntu virtual server. What you have learned here to schedule a simple script can also be used to schedule more complicated scripts. The script's difficulty does not matter while scheduling a task; creating a script schedule is relatively straightforward.

#	Task
---	------

- 1 First, make a new directory to place your script in, write the three-line Python code for `crontab` testing, and save it as `print_hello_friend.py`. You can change the greeting in your script. Here we will import the `datetime` module from the `datetime` library and put the `print (datetime.now())` statement before printing `hello`. Since I am from Australia, my favorite greeting is "G'day mate."

```
pynetauto@ubuntu20s1:~$ mkdir my_cron
pynetauto@ubuntu20s1:~$ cd my_cron
pynetauto@ubuntu20s1:~/my_cron$ nano
print_hello_friend.py
pynetauto@ubuntu20s1:~/my_cron$ cat
print_hello_friend.py
```

Task

```
from datetime import datetime
print(datetime.now())
print("G'day Mate!")
```

Next, run the code once to check that it prints out the time and your “hello” statement.

```
2 pynetauto@ubuntu20s1:~/my_cron$ python3
print_hello_friend.py
2021-01-13 14:01:40.770547
G'day Mate!
```

Now you are going to use `crontab` to schedule this Python code to run. Before you can create a schedule, you are going to learn how to use `crontab` quickly. First, you have to choose whether you want to run the script as a standard user or a root user. Second, decide whether you want to run your Python code in a nonexecutable file mode or change it to executable and then run it. You have already learned how to turn a nonexecutable file into an executable on a Linux server. If you have decided to run `crontab` as a nonroot Linux user, you must execute the `crontab -e` command with the leading `sudo` command so that you can type in `sudo crontab -e`. On the other hand, if you have decided to run `crontab` as the root user, you do not need to affix the `sudo` command.

3 a. non-root user `crontab` scheduler execution

```
pynetauto@ubuntu20s1:~/my_cron$ sudo crontab -e
```

b. root user `crontab` scheduler execution

```
pynetauto@ubuntu20s1:~/my_cron$ crontab -e
```

To make `cron` to execute your script correctly, you need to check the Python executable file location and run the `which python3` or `which python3.8` command to confirm the directory. Also, run the `pwd` command to check your working directory for your Python script.

```
4 pynetauto@ubuntu20s1:~/my_cron$ which python3.8
/usr/bin/python3.8
pynetauto@ubuntu20s1:~/my_cron$ pwd
/home/pynetauto/my_cron
```

5 Next, run `crontab` using the `crontab -e` command to open the `crontab` scheduler. The `pynetauto` user is a `sudoer` to run the command without the `sudo` command. When you use the `crontab -e` command for the first time, a message will pop up to select the text editor of your choice. Choose the easiest or the recommended one, which is a nano text editor, 1.

```
pynetauto@ubuntu20s1:~/my_cron$ crontab -e
no crontab for pynetauto - using an empty one
```

Task

Select an editor. To change later, run 'select-editor'.

1. /bin/nano <---- easiest
2. /usr/bin/vim.basic
3. /usr/bin/vim.tiny
4. /bin/ed

Choose 1-4 [1]: 1

After selecting nano as your text editor for the `crontab` scheduler, press the Enter key to open the `crontab` scheduler file, as shown next. Quickly browse the information and move your cursor to the bottom line of the file. Type in exactly what is shown here and save the file. You will learn more about `cron` time formats later, so you do not know what each line or each character means. For now, you must try it first with your own hands.

```
6 [...]
# For more information see the manual pages of
# crontab(5) and cron(8)
#
# m h dom mon dow command
* * * * * /usr/bin/python3.8
/home/pynetauto/my_cron/print_hello_friend.py >>
/home/pynetauto/my_cron/cron.log
```

Next, wait for one to two minutes. You should see that a logging file called `cron.log` is automatically created under your `/home/pynetauto/my_cron` directory.

```
pynetauto@ubuntu20s1:~/my_cron$ ls -lh
total 8.0K
-rw-rw-r-- 1 pynetauto pynetauto 39 Jan 13 14:09
cron.log
-rw-rw-r-- 1 pynetauto pynetauto 73 Aug 27 22:48
print_hello_friend.py
```

- 7 Check the `cron.log` file with the `cat` command to check if the `crontab` is running normally. You will see that the `cron` job is running every minute, and the greeting is recorded in the `cron.log` file. So, now you have learned that the five stars (* * * * *) in the `crontab -e` meant every minute.

```
pynetauto@ubuntu20s1:~/my_cron$ cat cron.log
2021-01-13 14:09:01.393031
G'day Mate!
2021-01-13 14:10:01.423129
G'day Mate!
```

- 8 To check your `cron` job, use the `crontab -l` command. Specify the user is an option if you are already logged in as the user.

Task

There is no `cron` job under the `root` user, so use it to check another user's `cron` schedule.

```
pynetauto@ubuntu20s1:~/my_cron$ crontab -l
* * * * * /usr/bin/python3.8
/home/pynetauto/my_cron/print_hello_friend.py >>
/home/pynetauto/my_cron/cron.log
```

```
pynetauto@ubuntu20s1:~/my_cron$ sudo crontab -u root -l
[sudo] password for pynetauto:*****
no crontab for root
```

9

To manage another user's `cron` job, you can run `sudo crontab -u user_name -l`. To modify time intervals or cancel another user's `cron` job, use `sudo crontab -u user_name -e` and make the required time changes. If the user does not have any `cron` job, it will return the `no crontab for user_name` message.

```
pynetauto@ubuntu20s1:~$ sudo crontab -u pythonadmin -e
pynetauto@ubuntu20s1:~$ sudo crontab -u pythonadmin -l
no crontab for pythonadmin
```

10

Finally, to cancel (deactivate) the currently running `crontab` schedule, open the scheduler file with `crontab -e`, add `#` in front of a specific `cron` task line, delete the line in question, and save the file. Another way to control `crontab`'s `cron` job is to stop/start/restart the `crontab` service by using the `service cron stop`, `service cron start`, and `service cron restart` commands.

```
# * * * * * /usr/bin/python3.8
/home/pynetauto/my_cron/print_hello_friend.py >>
/home/pynetauto/my_cron/cron.log
```



In step 6, each time `cron` job runs and executes the script, the activity is logged to the `cron.log` file. But, what do the two right arrows or greater-sign symbols (`>>`) mean here? What if a single arrow was used? Let's check the difference.

```
* * * * * /usr/bin/python3.8
/home/pynetauto/my_cron/print_hello_friend.py >>
/home/pynetauto/my_cron/cron.log
```

Using two right arrows appends the logs in the next line. So, the previous content is not overwritten.

```
pynetauto@ubuntu20s1:~/my_cron$ cat cron.log
```

```
...
```



```
2021-01-13 14:39:01.318529
```

```
G'day Mate!
```

```
2021-01-13 14:40:01.348718
```

```
G'day Mate!
```

```
...
```

If only one right arrow was used as shown here, the previous content would be overwritten entirely, and only the last executed content would be recorded to the file. Try it for yourself and check it out with your own eyes.

```
* * * * * /usr/bin/python3.8
/home/pynetauto/my_cron/print_hello_friend.py >
/home/pynetauto/my_cron/cron.log
```

```
pynetauto@ubuntu20s1:~/my_cron$ cat cron.log
```

```
2021-01-13 14:44:01.467980
```

```
G'day Mate!
```

CentOS8.1 Task Scheduler: crond

There are some minor differences between the CentOS and Ubuntu `cron` versions. You have to become familiar with both Red Hat and Debian-derived Linux OSs in production as they are most commonly found Linux OS in the enterprise IT ecosystems. Similar to Ubuntu's `crontab`, CentOS offers a `cron` called `crond`. Follow the instructions here in your CentOS 8.1 server to learn how to use `crond`. To complete the following tasks, log into `centos8s1` via an SSH connection. You will need some or all routers and switches running in your lab topology. See Figure [15-10](#).

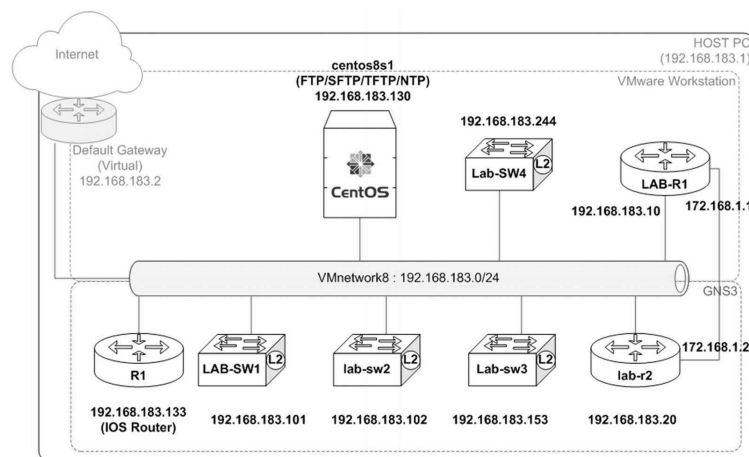


Figure 15-10. centos8s1, crond lab required equipment

#	Task
---	------

- 1 In the earlier chapter, you were introduced to probing a specific port (port 22); here, you will learn how to ping an IP address from your Linux server using a Python script. Let's create a simple ICMP pinging script and make it ping all IP addresses in our network and log it to the `cron.log` file. This time, we will create an external text file called `ip_addresses.txt` to read each device's IP addresses and then ping three times and record the activity to the `cron.log` file.

First, create two files, `ip_addresses.txt` and `ping_sweep1.py`, following these instructions:

```
[pynetauto@centos8s1 ~]$ mkdir icmp_sweeper
[pynetauto@centos8s1 ~]$ cd icmp_sweeper
[pynetauto@centos8s1 icmp_sweeper]$ touch
ip_addresses.txt
```

```
[pynetauto@centos8s1 icmp_sweeper]$ nano
```

```
ip_addresses.txt
```

```
ip_addresses.txt
```

```
192.168.183.10
```

```
192.168.183.20
```

```
192.168.183.101
```

```
192.168.183.102
```

```
192.168.183.133
```

```
192.168.183.153
```

```
192.168.183.244
```

Create a `ping_ip_add.py` file and start writing your code. It is important that you type this code yourself to get enough practice.

```
[pynetauto@centos8s1 icmp_sweeper]$ nano
```

```
ping_sweep1.py
```

```
ping_sweep1.py
```

```
#!/usr/bin/python3 # shebang line to tell Linux to run
this file as Python3 application
```

```
import os # import os module
```

```
import datetime # import datetime module
```

```
with
```

```
open("/home/pynetauto/icmp_sweeper/ip_addresses.txt",
"r") as ip_addresses: #with open closes file automati-
cally, read the file from specified directory
```

```
print("-"*80) # Divider
```

```
print(datetime.datetime.now()) # Print current time
```

```
for ip_add in ip_addresses: # Use a simple for loop
through each line in a file to get ip
```

```
ip = ip_add.strip() # Remove any white spaces
```

```
rep = os.system('ping -c 3 ' + ip) # Use Linux OS
```

```
to send 3 ping (ICMP) messages
```

```
if rep == 0: # response 0 means up
```

```
print(f"{ip} is reachable.") # Informational
```

Task

```
else: # response is not 0, then there maybe network
connectivity issue
```

```
print(f"{ip} is either offline or icmp is fil-
tered.") # Informational
```

```
print("-"*80) # Divider
```

```
print("All tasks completed.")
```

The previous script will print the current time and then ping each IP address three times. Then if it is on the network, it will print "IP is reachable." Otherwise, it will print "IP is either off-line or icmp is filtered."

This time, make the Python script an executable file so the code is simplified when scheduling in `crond`. First, check the user permissions for the script. Use the `chmod +x` command to make the file executable and check the permission again. The file's color should turn green after running the `chmod` command and x's added to the file attributes.

```
[pynetauto@centos8s1 icmp_sweeper]$ ls -l
total 8
-rw-rw-r--. 1 pynetauto pynetauto 110 Jan 13 15:03
ip_addresses.txt
2 -rw-rw-r--. 1 pynetauto pynetauto 954 Jan 13 15:02
ping_sweep1.py
[pynetauto@centos8s1 icmp_sweeper]$ chmod +x
ping_sweep1.py
[pynetauto@centos8s1 icmp_sweeper]$ ls -l
total 8
-rw-rw-r--. 1 pynetauto pynetauto 110 Jan 13 15:03
ip_addresses.txt
-rwxrwxr-x. 1 pynetauto pynetauto 954 Jan 13 15:02
ping_sweep1.py
```

- CentOS's `crond` has two noticeable differences from Ubuntu's `crontab`. The first obvious difference is the name, and also, when you have to reinstall `cron` on CentOS, you have to use the name `sudo yum install cronie` to install `crond`. You can use the `rpm -q cronie` command to check the `crond` version on CentOS.

```
[pynetauto@centos8s1 icmp_pinger]$ rpm -q cronie
cronie-1.5.2-4.el8.x86_64
```

The second difference is where the `cron` job is scheduled. Unlike Ubuntu, CentOS's `crond` schedules tasks directly in the `/etc/crontab` file, as shown here:

```
[pynetauto@centos8s1 icmp_sweeper]$ cat /etc/crontab
HELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
# For details see man 4 crontabs
```

```
# Task

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr
...
# | | | | .---- day of week (0 - 6) (Sunday=0 or
7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
```

Next, use the `systemctl status crond` command to check whether the `crond` service is operational.

```
[pynetauto@centos8s1 icmp_sweeper]$ systemctl status crond

● crond.service - Command Scheduler
   Loaded: loaded
   (/usr/lib/systemd/system/crond.service; enabled; vendor prese>
4 Active: active (running) since Wed 2021-01-13
14:55:14 AEDT; 19min ago
   Main PID: 1405 (crond)
     Tasks: 6 (limit: 11166)
    Memory: 4.4M
   CGroup: /system.slice/crond.service
           └─1405 /usr/sbin/crond -n
[...omitted for brevity]
```

- 5 When scheduling a shell command or Python script to run on CentOS `crond`, you can configure and schedule `cron` commands in different formats. For example, just as in the Ubuntu example earlier, you could use a full path method to schedule a `cron` job. In CentOS 8.1, the same method will work, as shown here:

```
[pynetauto@centos8s1 icmp_sweeper]$ which python3.6
/usr/bin/python3.6

[pynetauto@centos8s1 icmp_sweeper]$ pwd
/home/pynetauto/icmp_pinger

[pynetauto@centos8s1 icmp_sweeper]$ sudo nano
/etc/crontab

[... omitted for brevity]

*/5 * * * * root /usr/bin/python3.6
/home/pynetauto/icmp_sweeper/ping_sweep1.py >>
/home/pynetauto/icmp_sweeper/cron.log 2>&1
```

Here we are making the Python script an executable file and running it without specifying the application location. Your Python script must have the first shebang line (`#!/usr/bin/python3`) included in the script. Also, note that you have to specify a user to run the script; since `/etc/crontab` is a

Task

root user file, it is easiest to run the script as the root user, so when you schedule the task, make sure that the `root` user is specified for the `cron` job to run successfully. Now, open `etc/crontab` and schedule your ICMP script to send pings to all routers and switches in your lab every five minutes. `*/5 * * *` `*` denotes every fifth minute. The script will run at 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, and again 0 or 60th minutes until you pause or remove the `cron` job.

```
[pynetauto@centos8s1 icmp_pinger]$ sudo nano
```

```
/etc/crontab
```

```
[... omitted for brevity]
```

```
*/5 * * * * root
```

```
/home/pynetauto/icmp_sweeper/ping_sweep1.py >>
```

```
/home/pynetauto/icmp_sweeper/cron.log 2>&1
```

- 6 Finally, open the `cron.log` file under the current working directory and check that the communication check task is taking place every five minutes. You may have to wait for five minutes for the `cron.log` file to appear in your working directory, and when you view the file, it would look similar to this:

```
[pynetauto@centos8s1 icmp_sweeper]$ ls -lh
```

```
total 12K
```

```
-rw-r--r--. 1 root      root      3.2K Jan 13 15:35
```

```
cron.log
```

```
-rw-rw-r--. 1 pynetauto pynetauto 110 Jan 13 15:03
```

```
ip_addresses.txt
```

```
-rwxrwxr-x. 1 pynetauto pynetauto 886 Jan 13 15:35
```

```
ping_sweep1.py
```

```
[pynetauto@centos8s1 icmp_pinger]$ cat cron.log
```

```
PING 192.168.183.10 (192.168.183.10) 56(84) bytes of data.
```

```
64 bytes from 192.168.183.10: icmp_seq=1 ttl=255
```

```
time=15.8 ms
```

```
64 bytes from 192.168.183.10: icmp_seq=2 ttl=255
```

```
time=15.8 ms
```

```
64 bytes from 192.168.183.10: icmp_seq=3 ttl=255
```

```
time=21.9 ms
```

```
--- 192.168.183.10 ping statistics ---
```

```
3 packets transmitted, 3 received, 0% packet loss,
```

```
time 6ms
```

```
rtt min/avg/max/mdev = 15.768/17.798/21.850/2.865 ms
```

```
[... omitted for brevity]
```

```
PING 192.168.183.244 (192.168.183.244) 56(84) bytes of data.
```

```
64 bytes from 192.168.183.244: icmp_seq=2 ttl=255
```

```
time=8.33 ms
```

```
64 bytes from 192.168.183.244: icmp_seq=3 ttl=255
```

```
time=7.81 ms
```

```
# Task

--- 192.168.183.244 ping statistics ---
3 packets transmitted, 2 received, 33.3333% packet
loss, time 7ms
rtt min/avg/max/mdev = 7.808/8.070/8.333/0.277 ms
-----

-----

2021-01-13 15:35:01.973235
192.168.183.10 is reachable.
192.168.183.20 is reachable.
192.168.183.101 is reachable.
192.168.183.102 is reachable.
192.168.183.133 is reachable.
192.168.183.153 is reachable.
192.168.183.244 is reachable.
-----

-----

All tasks are completed.
```

At the end of this lab, make sure you go back to the
7 /etc/crontab file and add # to the beginning of each line to de-
activate it. You can optionally delete the line and then save the
changes.



Out of the box, `crontab` is much easier to use than `crond`. Also, it has the flexibility of scheduling `cron` jobs under different user accounts, so many users will find `crontab` more intuitive to use. If you are like me, you can choose to install `crontab` on the CentOS8.1 server using the following Linux commands. But note that, in the production environment, some old Linux servers are not connected to the Internet, and you may have to use `crond` to schedule the tasks, so it is always good to know the difference between different `cron` tools on different Linux operating systems. Anyway, to install `crontab` on your CentOS8 VM, follow these commands:

```
[pynetauto@centos8s1 ~]$ sudo dnf update
```

```
[pynetauto@centos8s1 ~]$ sudo dnf install crontabs
```

```
[pynetauto@centos8s1 ~]$ sudo systemctl enable crond.service
```

```
[pynetauto@centos8s1 ~]$ sudo systemctl start crond.service
```

Noted `dnf` (aka the next-generation package management utility for RPM-based distributions) will be replacing old `YUM` package management utilities in the Fedora, CentOS, and Red Hat distributions.

Next, let's review the `cron` job definitions or the five asterisks.

Learn cron Job Definitions with Examples

To use Linux `cron` effectively, you must understand the meaning of the five stars (asterisks) at the beginning of each `cron` job line. If you are an existing `cron` user and understand what five asterisks denote and how they are used, that is great; then you can move to the next section. If you are new to `cron`, please read the rest of this section to grasp the meaning of the five asterisks. The following snippet is straight from the `/etc/crontab` file from CentOS; this is all you need to know to understand how a command, script, or task is scheduled on `cron` :

```
# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name  command to be executed
```

For ease of explanation, all exercises will be based on `crontab` rather than `crond`, so open your `crond` in Ubuntu 20 LTS.

Generally, a default `cron` job begins with five stars (asterisks). These five stars relate to the frequency of task run, or, in other words, the time and date interval of a certain task. After the five values is the command (CMD) required to run a specific task. Here’s an example:

Minute	Hour	Day of month	Month	Day of week	CMD
0	1	31	12	*	/home/pynetauto/ny_eve.py >> ~/cron_time.log

Table **15-1** is the tabulated version of the example of a job definition shown earlier .

Table 15-1. Linux crontab: Time Unit

Unit	Available Values	Conversion
Min of an hour	0–59 minutes	1 hour = 60 minutes
Hour of a day	0–23 hours	1 day = 24 hours
Day of month	1–31 days	1 month = 28–31 days
Month	1–12 or Jan, Feb, Mar, Apr... Dec	Jan, Feb, Mar, Apr...
Day of week	0–6	1 week = 7 days
CMD	Command to run	N/A

Depending on your Linux OS type, each command also has slightly different variations, but they are almost identical for the most part. If these values use

another format, you should refer to the OS distributor's documentation for different time unit variations. Let's study some more examples to get an understanding of the `cron` schedule's job definitions. The following examples cover the most common job definitions. If you have a different requirement, you can make variations to the following examples and use them in your `cron` job scheduling.

#	Examples (Alternatives in Brackets)	Explanation
1	* * * * *	The five asterisks are the default value, which tells the Linux system to execute a task every minute. You have used the five asterisks to run a Python script already.
2	15 03 25 03 *	At 3:15 on the 25th day of March.
3	15 1,13 * * *	At 1:15 a.m. and 1:15 p.m.
4	00 07-11 * * *	On the hour between 7 a.m. to 11 a.m. (every day)
5	00 07-11 * * 1-5	Same as example 4, but only run on the weekdays (1 = Mon, to 5 = Fri)
6	0 2 * * 6,0	At 2 a.m. on Saturdays and Sundays only
7	00 23 * * 1 (00 23 * * Mon)	At 11 p.m. on Mondays
8	0/5 * * * *	At every 5 th minute, every 5 minutes
9	0/15 * * * *	At every 15 th minute, every quarter of an hour
10	0/30 * * * *	At every 30 th minute, every half an hour
11	0 0 * 1/1 *	At midnight on the first day of each month
12	0/10 * * * * 0 (0,10,20,30,40,50 * * * 0)	At every 10 th minute; supported on newer <code>cron</code> versions At every 10 th minute; supported on older <code>cron</code> versions
13	@reboot	After rebooting

#	Examples (Alternatives in Brackets)	Explanation
14	@hourly (0 * * * *)	At minute 0
15	@daily (@midnight) (0 0 * * *)	At 00:00 (midnight)
16	@weekly (0 0 * * 0)	At midnight every Sunday
17	@monthly (0 0 1 * *)	At midnight, first day of the month
18	@yearly (@annually) (0 0 1 1 *)	At midnight on January 1 st every year

 A more detailed description of `cron` can also be found on Wikipedia. You can learn more about `cron` and how to use `cron` by visiting the following Wikipedia site .

URL: <https://en.wikipedia.org/wiki/Cron>

And, you can learn more about how to use `cron` practically by using one of the `cron` maker sites listed here:

URL: <http://www.cronmaker.com/>

URL: <https://crontab.guru/>

URL: <http://corntab.com/>

In this section, you configured and completed scheduling for simple Python scripts using Linux `cron`, and then you learned the meanings behind the five stars using examples. For now, you do not have to comprehend the `cron` definitions by heart; you can always come back to this page as a reference. Next, you will be revisiting SNMP basics to get you ready for Python interaction with SNMPv3 on network devices and discuss how Python can be used for system monitoring using SNMPv3.

Using Python to Run an SNMPv3 Query

Although SNMP is used everywhere for enterprise network monitoring, it is an under-appreciated technology that keeps checking our network all the time. Not many network engineers pay much attention to how SNMP works in depth or what it does because it works seamlessly. We really cannot blame engineers for being ignorant about SNMP as network moni-

toring is considered an entry-level engineer job. When you move up the ladder, you focus more on the client services as the number-one priority and then the design, implementation, and consulting parts of network administration. But, should you know how Python interacts with SNMP? The answer is yes!

In this section, you will quickly review SNMP concepts to get you up to speed and then borrow some Python SNMPv2c code to rewrite the code to interact with the network devices using SNMPv3. A full Python tools development using SNMP may take weeks or months, so here you will simply dip in your toes to learn how Python interacts with network devices using SNMP.

Quick-Start Guide to SNMP

To help you refresh your SNMP knowledge, we will be going through some basic SNMP facts. If you have been working with SNMP technology every day, you can skip this part and jump to the SNMP lab. If you are not familiar with SNMP, this will be a quick-start guide to SNMP to learn how Python interacts with SNMP.

Quick SNMP History

To get a bigger picture of a particular technology, it is always good to know a technology's history. SNMP began its life as ICMP and evolved until it became the monitoring tool of choice.

- *1981*: The Internet Engineering Task Force (IETF) defined the Internet Control Management Protocol (ICMP) , which is a protocol used to probe the network communication status of devices on the network.
- *1987*: ICMP was redefined as the Simple Gateway Monitoring Protocol (SGMP) as part of RFC 1028.
- *1989 to 1990*: SGMP was redefined as the Simple Network Monitoring Protocol (SNMP) , which was introduced as part of RFC 1155, 1156, and 1157.
- *1990 to present*: Most IT and networking vendors accepted SNMP as the standard protocol to monitor IP devices.

Source: <https://tools.ietf.org/html/> (RFC: 777, 1028, 1155, 1156, 1157)

SNMP Standard Operation

As in many server and client model , SNMP also uses server and client model. The SNMP server is known as the *manager*, and it works with clients (nodes), known as *agents*. Even if you have completed your CCNA studies, you may not be familiar with the ins and outs of standard SNMP operations. If this is your first time using SNMP, don't skip this section .

- SNMP works as the server (manager) and client (agent).
- Generally, the SNMP agent is installed on monitored devices as agents.
- SNMP follows the preset rules to monitor network devices and collect and store information to monitor devices.
- An agent can send its status to the manager using either polling or event reporting methods.

- Management Information Base (MIB) is the information standard used by the agent to collect data.
- Object identification (OID) is the primary key of MIB's management object.

SNMP Polling vs. Trap (Event Reporting) Method

To collect data, the SNMP manager sends the Get request message to the agent, and then the agent sends back a response message with its status. The SNMP server requests specific information, and then the SNMP client responds. In an environment where the SNMP server can change the SNMP client's status, the server can use the SNMP Set request to change the SNMP client's status. In other words, you can use SNMP to control the client, but usually, SNMP's function is limited to monitoring the network. The polling method is talkative on the network, so it generates more traffic than the Trap method. In the Trap method, the SNMP client sends information to the SNMP server's Trap receiver, only if there was any monitored information in its system. This is the most common network monitoring in the industry today. Also, it generates a relatively small amount of network traffic in a stable network.

SNMP Versions

Table **15-2** shows all the SNMP versions in action. Many legacy systems do not support the more secure SNMPv3, so in most networks, SNMPv2c commonly known as SNMPv2 is used the most, but due to security issues, more and more enterprises are moving toward SNMPv3. As the old devices get refreshed, more devices can support SNMPv3, and it is slowly getting deployed as the legacy devices are decommissioned. Another significant catalyst to move away from SNMPv2 is the introduction of virtualized and cloud-based infrastructures that are now the norm of enterprise network and server networking. In a virtualized cloud world, besides the actual server farm and physical connections, almost everything exists as files or software, meaning that technologies are not anchored down to their hardware limitations. Review Table **15-2** to check the features of each version .

Table 15-2. SNMP Versions

Version	Features
SNMPv1	Uses a community string (key); no built-in security; only supports 32-bit systems
SNMPv2c	Most commonly used; supports 32/64-bit systems; improved performance for 64-bit systems; almost identical features to v1; low security; cross-platform support
SNMPv3	Manager and agent system changed to objects; increased 64-bit system security; guarantees authentication and privacy; detects each SNMP entities as unique EngineID names

Understanding the SNMP Protocol in the TCP/IP Stack

SNMP is an application protocol in the TCP/IP protocol stack and is sitting on the transport layer in the OSI model; it uses UDP to communicate between the server

and the client. Here is a quick summary of SNMP communication port information and how the manager and agents communicate:

- The SNMP Get message communicates via UDP port 161.
- The SNMP Set message also communicates via UDP port 161.
- When SNMP Trap is used, SNMP Trap communicates via UDP port 162, and the agent that sends the message does not have to check if the trap was sent correctly.
- When SNMP Inform is used, it behaves the same way except it confirms that the Inform message has been received correctly via UDP port 162.

About SNMP Message Types

There are six types of messages sent and received between the SNMP manager and SNMP agent; these message types are summarized in Table [15-3](#).

Table 15-3. SNMP Message Types

Message Type	PDU Type	Explanation
get-request	0	Gets the value of the object specified in the agent management MIB.
get-next-request	1	Gets the next object value of the object specified in the agent management MIB. It is used when the MIB item has a table.
get-bulk-request	1	It works like <code>get-next-request</code> and gets the object's value specified in the agent management MIB as many times as specified.
set-request	2	Sets (changes) the value of the object specified in the agent management MIB.
get-response	3	Returns the result of the manager's request.
traps	4	This message is sent to the manager to inform the manager when a particular event occurs in the agent management MIB.

**PDU type values range between 0 and 4 only.*

Understanding SNMP's SMI, MIB, and OID

Next, after briefly explaining the definition of SMI, MIB, and OID of SNMP, you will take a look at security methods, access policies, synchronization, and asynchronization of traps. Let's quickly cover SNMP terminologies and what purpose they serve in SNMP monitored networks.

- *Structured Management Information (SMI)*: SMI is a tool for defining and configuring MIBs and creating and managing MIB standards.

- *Management Information Base (MIB)*: The MIB is a set of objects managed from the SNMP server's perspective. The MIB is a database of objects that can be viewed or set by an administrator. Each object has a tree-like structure, and you can check the status or change the value of an agent with specific MIB values. ASN.1 (Abstract Notation One), all organized in a tree structure, is a scripting language for describing objects, including SNMP objects.
- *Object-Identifier (OID)*: OID must be assigned to generate the MIB. OIDs are represented as a sequence of unique, specific numbers, using a notation similar to IP addresses. OID is an identifier (primary key) that designates the administered object of the MIB. You can run OID lookup tools to gain an understanding of OID in detail.

SNMP Access Policy

To ensure SNMP security, SNMP community strings are used between manager and agent communication. The manager requires the appropriate community string to access key device information across your network. Most network vendors configure the default public community string as public when the network equipment leaves the factory; many network administrators change the community string to keep intruders from getting information about the network setup. SNMP can offer different access levels to information based on different community strings. There are Read-Only, Read-Write, and Trap community strings

- *SNMP Read-only community string*: The agent's information can be read from other SNMP managers. `InterMapper` maps devices using read-only information.
- *SNMP Read-Write community string*: This is used for communication using requests and can read and change other devices' status or settings. Note that the `InterMapper` is read-only, so it does not use the read-write string mode.
- *SNMP Trap community string*: This is the community string used by the agent to send SNMP traps to the `InterMapper`, which receives and stores information no matter which SNMP Trap community string is used.

Synchronous vs. Asynchronous Communication While Using SNMP Traps

The following is a brief description of two SNMP Trap methods used by SNMP. In general, if the trap message is communicated between the SNMP manager (server) and the agent (client) using SNMP, the SNMP must use either synchronous or asynchronous mode. The differences between these two modes are explained here:

- *Synchronous mode*: While using synchronous mode, the sending and receiving sides use one reference clock separately from the data. The manager and the agent must operate together according to the synchronization signal. To perform continuous data transmission, it is a mode in which one data transmission/reception must be completed between the sending side and the receiving side to transmit the next data.
- *Asynchronous mode*: In the case of using the asynchronous mode, the time difference is identified with the received signal clock and trans-

mitted one character at a time regardless of the sender's clock. The start bit and stop bit are added to the front of the packet to be sent. In other words, in the case of the SNMP agent operating in the asynchronous mode, various data is exchanged when communicating with Get requests, and irregular transmission may be possible regardless of time and order.

When using SNMP Trap , the message is generated as a general trap or inform trap message. The trap uses an asynchronous communication method. The SNMP agent unilaterally sends information that does not require confirmation of the receiver, the SNMP manager. On the contrary, the SNMP agent works in the synchronous mode because the information sent must be acknowledged by the receiver, which is the SNMP manager.

SNMP-Related Python Libraries

For Python to use SNMP to communicate with other network devices, you must first install the correct SNMP library to help you achieve your goal. In this book, the pysnmp module will be installed to interact with routers and switches using Python. But there are several SNMP Python modules you can utilize based on what you want to achieve with Python and SNMP. Table 15-4 describes the Python SNMP-related modules for your future reference.

Table 15-4. Python SNMP Libraries

SNMP Package Name	Features and Download Link
pysnmp	Python-based SNMP module Program execution speed is slow https://pypi.org/project/pysnmp/
python-netsnmp	Use default Python bindings for net-snmp Supports non-Pythonic interface http://net-snmp.sourceforge.net/wiki/index.php/Python_Bindings
snimpy	Python tool developed for SNMP query Developed based on PySNMP Program execution speed is slow https://snimpy.readthedocs.io/en/latest/
easy-snmp	Library developed based on net-snmp Python-style interface support and operation Program execution speed is fast https://easysnmp.readthedocs.io/en/latest/
fastsnmpy	Using an asynchronous wrapper based on net-snmp Developed for SNMPwalk https://github.com/ajaysdesk/fastsnmpy

Understanding MIBs and OIDs and Browsing OIDs Directly

The SNMP agent shares a lot of information with the SNMP manager. Let's get familiar with MIBs and OIDs. As discussed earlier, OID is an identifier that designates an administered object in the MIB. This means that OID is part of MIB, and it uses a unique ID to represent each MIB. An example of OID is 1.3.6.1.2.1.1.3.0; this OID is an identifier that contains information about the uptime after a system has booted up. These unique numbers for each SNMP can be found through the MIB library, and equipment manufacturers share this information through their official websites. If the device is connected to the network, both the systems and network devices can communicate their system status via SNMP using MIBs and unique OIDs. Here, you will use an interactive Linux session to perform `snmpwalk` on Cisco CML routers and switches in our GNS3 lab. Later, you will find working SNMPv2c Python code from the Internet and change it to support SNMPv3 and probe device information within our lab topology.

Learn to Use SNMPwalk on a Linux Server

First, you need to install SNMP software to make your Linux server an SNMP manager. Then, set a password to authenticate with the manager depending on which agents use the SNMP version. In general, the SNMP manager is installed as an independent server on a Linux or Windows server, and a program that supports the SNMP API is installed and used. The most commonly used SNMP server programs are SolarWinds Network Performance Monitor, Paessler PRTG Network Monitor, and SysAid Monitoring. Most of this software provides a free distribution version for testing use. For more detailed information on these programs, please refer to each vendor's website and resources.

Here, we will install the SNMP server software on a Linux server, configure SNMPv3 on all network devices, and then execute the OID of the virtual network devices' MIB at the command line. First, install the SNMP manager software on the Linux server as described in the following section. If you want to monitor from another SNMP server, you can choose to install the SNMP agent (client) software. The Linux server used is a CentOS 8.1 server, so you will have to be log into the `centos8s1` server to follow the tasks step by step. The lab topology is the same as before; you will need a CentOS server and all routers and switches powered on.

Installing and Configuring SNMP Server on CentOS8

Follow these steps to install SNMP services on your CentOS 8 server and configure the user and user's authentication and authorization settings. You have to type in the commands in your `centos8s1` server that are marked in bold.

#	Task
---	------

1	First, install the <code>net-snmp-utils</code> and <code>net-snmp-devel</code> packages on the CentOS server using the commands shown here. <code>net-snmp-utils</code> is a tool used for <code>snmpwalk</code>. <pre>[pynetauto@centos8s1 ~]\$ sudo yum install -y net-snmp net-snmp-utils net-snmp-devel</pre>
---	--

Task

- 2 Configure an SNMPv3 user and an authentication password . The information configured here will be used to configure SNMP agents, network devices in this case. First, enable and start `snmpd` on the CentOS server; then check the running status. After that, stop the service before configuring the new SNMP user and password.

```
[pynetauto@centos8s1 ~]$ sudo systemctl enable snmpd
[pynetauto@centos8s1 ~]$ sudo systemctl start snmpd
[pynetauto@centos8s1 ~]$ sudo systemctl status snmpd
[pynetauto@centos8s1 ~]$ sudo systemctl stop snmpd
```

There are three methods to configure an SNMPv3 user with a password on a Linux system, and all three methods will be discussed here. If authentication and encryption methods are not specified during the user configuration step, MD5 and AES will be used automatically, and the access level will be set to read-only (`-ro`).

Use the one of the methods to add an SNMP user and passphrase.

A. Command-line method: If you set up a user via a command, you can simultaneously set the username, authentication password, and encryption-only key in one command line. Typical command options look like this:

```
net-snmp-config --create-snmpv3-user [-ro] [-A authpass] [-X privpass] [-a MD5|SHA] [-x DES|AES] [username]
```

Example 1: SNMP user creation without specifying a private password using default MD5 and DES:

```
[pynetauto@centos8s1 ~]$ sudo net-snmp-config --create-snmpv3-user -A AUTHPass1
SNMPUser2
```

adding the following line to `/var/lib/net-snmp/snmpd.conf`:

```
createUser SNMPUser1 MD5 "AUTHPass1" DES ""
```

adding the following line to `/etc/snmp/snmpd.conf`:

```
rwuser SNMPUser1
```

Example 2: SNMP user creation specifying authentication and encryption and a private password method. The following example configures MD5 authentication with DES encryption:

```
[pynetauto@centos8s1 ~]$ sudo net-snmp-config --create-snmpv3-user -ro -A
AUTHPass1 -X PRIVPass1 -a MD5 -x DES SNMPUser3
```

adding the following line to `/var/lib/net-snmp/snmpd.conf`:

```
createUser SNMPUser2 MD5 "AUTHPass1" DES "PRIVPass1"
```

adding the following line to `/etc/snmp/snmpd.conf`:

```
rouser SNMPUser2
```

Example 3: This is SNMP user creation using SHA authentication and AES encryption :

```
[pynetauto@centos8s1 ~]$ sudo net-snmp-config --create-snmpv3-user -ro -A
AUTHPass1 -X PRIVPass1 -a SHA -x AES SNMPUser1
```

adding the following line to `/var/lib/net-snmp/snmpd.conf`:

```
createUser SNMPUser3 SHA "AUTHPass1" AES "PRIVPass1"
```

adding the following line to `/etc/snmp/snmpd.conf`:

```
rouser SNMPUser3
```

B. Manual configuration: You can also set up SNMP users by opening the SNMP configuration file yourself and appending the new user information at the end of the file. Just open the two files and add the user information.

Example 4: This is a user account called `SNMPUser1` using MD5, and DES is created. Add the last line and username, as shown here:

```
[pynetauto@centos8s1 ~]$ sudo nano /var/lib/net-snmp/snmpd.conf
[...omitted for brevity]
```



```

# Task

#####

#
# snmpNotifyFilterTable persistent data
#
#####

engineBoots 18

oldEngineID 0x80001f8880acfa8434b71f4a5f00000000

createUser SNMPUser4 MD5 "AUTHPass1" DES "PRIVPass1" <<< Add this to the last
line
[pynetauto@centos8s1 ~]$ sudo nano /etc/snmp/snmpd.conf
[...omitted for brevity]
#####

# Further Information
#

# See the snmpd.conf manual page, and the output of "snmpd -H".

rwuser SNMPUser2
rouser SNMPUser3
rouser SNMPUser1

rwuser SNMPUser4 <<< Add this to the last line

Interactive mode: This interactive command is useful when creating SNMP users using MD5
and DES.

[pynetauto@centos8s1 ~]$ sudo net-snmp-create-v3-user
[sudo] password for pynetauto:
Enter a SNMPv3 user name to create:

SNMPUser5

Enter authentication pass-phrase:

AUTHPass1

Enter encryption pass-phrase:
[press return to reuse the authentication pass-phrase]

PRIVPass1

adding the following line to /var/lib/net-snmp/snmpd.conf:

    createUser SNMPUser MD5 "AUTHPass1" DES "PRIVPass1"

adding the following line to /etc/snmp/snmpd.conf:

    rwuser SNMPUser5

```

Task

Enter the following command to start the SNMP service again and check the status of the snmpd service . The status must be “Active: active (running).”

```
[pynetauto@centos8s1 ~]$ sudo systemctl start snmpd
```

```
[pynetauto@centos8s1 ~]$ sudo systemctl status snmpd
```

● snmpd.service - Simple Network Management Protocol (SNMP) Daemon.

Loaded: loaded (/usr/lib/systemd/system/snmpd.service; enabled; vendor preset:

3 disabled)

Active: active (running) since Wed 2021-01-13 16:23:03 AEDT; 5s ago

Main PID: 43244 (snmpd)

Tasks: 1 (limit: 11166)

Memory: 4.9M

CGroup: /system.slice/snmpd.service

└─43244 /usr/sbin/snmpd -LSO-6d -f

Use the `SNMPwalk` command to check if SNMP is working correctly on the local server. Run `snmpwalk` of the CentOS server using `SNMPUser` created in method C.

```
[pynetauto@centos8s1 icmp_sweeper]$ snmpwalk -v3 -u SNMPUser5 -l authPriv -a MD5 -A AUTHPass1 -X PRIVPass1 localhost
```

```
SNMPv2-MIB::sysDescr.0 = STRING: Linux centos8s1 4.18.0-193.14.2.el8_2.x86_64
```

```
#1 SMP Sun Jul 26 03:54:29 UTC 2020 x86_64
```

```
SNMPv2-MIB::sysObjectID.0 = OID: NET-SNMP-MIB::netSnmpAgentOIDs.10
```

```
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (15885) 0:02:38.85
```

```
SNMPv2-MIB::sysContact.0 = STRING: Root <root@localhost> (configure /etc/snmp/snmp.local.conf)
```

4 SNMPv2-MIB::sysName.0 = STRING: centos8s1

```
SNMPv2-MIB::sysLocation.0 = STRING: Unknown (edit /etc/snmp/snmpd.conf)
```

```
SNMPv2-MIB::sysORLastChange.0 = Timeticks: (0) 0:00:00.00
```

```
SNMPv2-MIB::sysORID.1 = OID: SNMP-FRAMEWORK-MIB::snmpFrameworkMIBCompliance
```

```
SNMPv2-MIB::sysORID.2 = OID: SNMP-MPD-MIB::snmpMPDCompliance
```

```
SNMPv2-MIB::sysORID.3 = OID: SNMP-USER-BASED-SM-MIB::usmMIBCompliance
```

```
SNMPv2-MIB::sysORID.4 = OID: SNMPv2-MIB::snmpMIB
```

```
SNMPv2-MIB::sysORID.5 = OID: SNMP-VIEW-BASED-ACM-MIB::vacmBasicGroup
```

```
SNMPv2-MIB::sysORID.6 = OID: TCP-MIB::tcpMIB
```

```
SNMPv2-MIB::sysORID.7 = OID: IP-MIB::ip
```

```
SNMPv2-MIB::sysORID.8 = OID: UDP-MIB::udpMIB
```

```
[... omitted for brevity]
```



The following is a brief description of the option handles used in the

Linux `snmpwalk` command :

snmpwalk -v3 -u SNMPUser2 -l authPriv -a MD5 -A AUTHPass1 -X PRIVPass1

127.0.0.1-v3 (version)

-u (SNMP username)

-l (security level)

-a (authentication method)

-A (pass-phrase)

Configuring Cisco Routers and Switches to Support SNMPv3 Using a Python Script

The following are the SNMP-related settings for Cisco devices. To implement SNMPv3 instead of SNMPv2c, you have to understand more about SNMP authentication than SNMPv2c. SNMP v2c still is the most popular SNMP version in use today due to the legacy compatibility issues of SNMPv3. Older devices do not have the software code to support the SNMP version3, so unless your network consists of old devices, SNMPv2c will be the way to go. In the future, almost all devices will support more secure SNMPv3. In this book, you are going to use SNMPv3 instead. You are going to write a quick Python script to add SNMP configurations on all Cisco devices.

Task

All SNMP agents manage their own (system) information separately, which can be used in SNMPv3 messages. For SNMP agent devices such as Cisco routers and switches, Cisco recommends setting unique SNMP engine IDs before configuring and using SNMPv3. If the SNMP engine ID is not specified during configuration, then a combination of the enterprise number and the default MAC address will generate a unique engine ID. To correctly add an SNMPv3 configuration to Cisco devices, you have to use three `snmp-server` commands as suggested here.

A. The engine ID configuration command example looks like this:

```
LAB-R1(config)# snmp-server engineID local 19216818310
```

B. The SNMP group name configuration command is shown next. `GROUP1` is the SNMP group name in this case. This command will create an SNMPv3 group called `GROUP1` with a private password.

```
LAB-R1(config)# snmp-server group GROUP1 v3 priv
```

1 C. Use the same information as the manager-side SNMP user credentials, and you can configure the SNMP user on a Cisco device using the following command :

```
LAB-R1(config)# snmp-server user SNMPUser1 GROUP1 v3 auth sha AUTHPass1
priv aes 128 PRIVPass1
```

You can use `show snmp user` to check the SNMP user information on Cisco devices. In the next step, based on the previous commands, let's write a Python script to add this configuration to all devices at once.

```
LAB-R1#show snmp user
```

```
*Jan 13 05:22:55.217: %SYS-5-CONFIG_I: Configured from console by console
User name: SNMPUser1
Engine ID: 192168183100
storage-type: nonvolatile          active
Authentication Protocol: SHA
Privacy Protocol: AES128
Group-name: GROUP1
```

2 To make this lab more enjoyable, we will copy the ICMP sweeper tool we created earlier and use it as the precommunication check tool in our script before SSHing into devices and configuring SNMP. So, use these commands to copy the files and then rename the `ping_sweep1.py` script as `precheck_tool.py`:

```
[pynetauto@centos8s1 ~]$ pwd
/home/pynetauto
```

Task

```
[pynetauto@centos8s1 ~]$ mkdir snmp_test
[pynetauto@centos8s1 ~]$ ls icmp_sweeper
cron.log ip_addresses.txt ping_sweep1.py
[pynetauto@centos8s1 ~]$ mkdir snmp_test
[pynetauto@centos8s1 ~]$ cp ./icmp_sweeper/ping_sweep1.py
./snmp_test/precheck_tool.py
[pynetauto@centos8s1 ~]$ cp ./icmp_sweeper/ip_addresses.txt
./snmp_test/ip_addresses.txt
[pynetauto@centos8s1 ~]$ cd snmp_test
[pynetauto@centos8s1 snmp_test]$ ls
ip_addresses.txt precheck_tool.py
```

You should have two files under the `/home/pynetauto/snmp_test` directory at the end of this task. The `ip_addresses.txt` file should already contain the IP addresses of all routers and switches in our topology.

You will open and modify `precheck_tool.py` in the nano text editor so that the main part of the script becomes a function. This way, we can call upon this function from another Python script as a module. Modify the file as follows:

```
[pynetauto@centos8s1 snmp_test]$ nano precheck_tool.py
GNU nano 4.8 /home/pynetauto/snmp_test/precheck_tool.py
#!/usr/bin/python3
import os

def icmp_pinger(ip):
    rep = os.system('ping -c 3 ' + ip)
    if rep == 0:
        print(f"{ip} is reachable.") # Print ip is reachable if the device is
on the network
    else:
3       print(f"{ip} is either offline or icmp is filtered. Exiting.")
        exit() # Exit application if any ip address is unreachable.
print("-"*80)

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C
Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_
Go To Line
```

When the `else` condition is met, for example, 192.168.183.10 is unreachable, it will exit the program as shown here:

```
[pynetauto@centos8s1 snmp_test]$ python3 snmp_config.py
PING 192.168.183.10 (192.168.183.10) 56(84) bytes of data.
--- 192.168.183.10 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 62ms
192.168.183.10 is either offline or icmp is filtered. Exiting.
```

- 4 Now, it is time for you to create the configuration script `snmp_config.py`. As a best practice, we will specify the SNMP engine ID using each device's IP address. The dots in IP addresses will be removed, and the whole number will be reused as the device's SNMP engine ID.

```
[pynetauto@centos8s1 snmp_test]$ nano snmp_config.py
```

Task

```

GNU nano 4.8 /home/pynetauto/snmp_test/snmp_config.py
#!/usr/bin/python3
import time
import paramiko
from getpass import getpass

# Custom tools
from precheck_tool import icmp_pinger

with open("/home/pynetauto/snmp_test/ip_addresses.txt", "r") as
ip_addresses:
    for ip in ip_addresses:
        ip = ip.strip()
        icmp_pinger(ip)

username = input("Enter username : ") # Ask for username
password = getpass("Enter password : ") # Ask for password
with open("/home/pynetauto/snmp_test/ip_addresses.txt", "r") as
ip_addresses:
    for ip in ip_addresses:
        ip = ip.strip()
        eng_id = ip.replace(".", "") # Remove dot in ip address and use it as
SNMP Engine ID

        print ("Now logging into " + (ip))
        ssh_client = paramiko.SSHClient()
        ssh_client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        ssh_client.connect(hostname=ip,username=username,password=password)
        print (f"Successful connection to {ip}\n")
        print ("Now completing following tasks : " + "\n")
        remote_connection = ssh_client.invoke_shell()
        print (f"Adding SNMP configuration to {ip}")
        remote_connection.send("show clock\n")
        remote_connection.send("configure terminal\n") # Add SNMP
configurations
        remote_connection.send(f"snmp-server engineID local {eng_id}\n")
        remote_connection.send("snmp-server group GROUP1 v3 priv\n")
        remote_connection.send("snmp-server user SNMPUser1 GROUP1 v3 auth sha
AUTHPass1 priv aes 128 PRIVPass1\n")
        remote_connection.send("do wri\n")
        remote_connection.send("exit\n")
        time.sleep(2)
        print ()
        time.sleep(2)
        output = remote_connection.recv(65535)
        print((output).decode('ascii'))
        print(f"Successfully configured {ip} & Disconnecting.")
        print("-"*80)
        ssh_client.close
        time.sleep(2)
        print("All tasks were completed successfully. Bye!")

```

```
# Task

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text   ^J Justify   ^C
Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell  ^_
Go To Line
```

Before running the script, you will need to install the `paramiko` library by running the

- 5 following `pip3` command. Install `paramiko` on the `centos8s1` server.

```
[pynetauto@centos8s1 snmp_test]$ sudo pip3 install paramiko
```

You should see the last line of code as a successful configuration script, which is outside of the `for` loop in the main script. Run `snmp_config.py` and configure SNMP group settings on all devices. The script will check the network connection using ICMP initially. If any device is unreachable, the application will exit. If all devices are on the network, the script will run and complete the SNMP configuration on all devices.

```
6 [pynetauto@centos8s1 snmp_test]$ python3 snmp_config.py
PING 192.168.183.10 (192.168.183.10) 56(84) bytes of data.
64 bytes from 192.168.183.10: icmp_seq=1 ttl=255 time=14.7 ms
64 bytes from 192.168.183.10: icmp_seq=2 ttl=255 time=11.5 ms
64 bytes from 192.168.183.10: icmp_seq=3 ttl=255 time=6.22 ms

--- 192.168.183.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 6ms
rtt min/avg/max/mdev = 6.215/10.819/14.749/3.516 ms
192.168.183.10 is reachable .
-----
-----
PING 192.168.183.20 (192.168.183.20) 56(84) bytes of data.
64 bytes from 192.168.183.20: icmp_seq=1 ttl=255 time=22.4 ms
64 bytes from 192.168.183.20: icmp_seq=2 ttl=255 time=25.0 ms
64 bytes from 192.168.183.20: icmp_seq=3 ttl=255 time=22.5 ms
[...omitted for brevity]
Successfully configured 192.168.183.153 & Disconnecting.
-----
-----
Now logging into 192.168.183.244
Successful connection to 192.168.183.244
Now completing following tasks :
Adding SNMP configuration to 192.168.183.244
*****
* IOSv is strictly limited to use for evaluation, demonstration and IOS *
* education. IOSv is provided as-is and is not supported by Cisco's *
* Technical Advisory Center. Any use or disclosure, in whole or in part, *
* of the IOSv Software or Documentation to any third party for any *
* purposes is expressly prohibited except as otherwise authorized by *
* Cisco in writing. *
*****
Lab-SW4#show clock
*12:33:19.576 UTC Wed Jan 13 2021
```

Task

```
Lab-SW4#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Lab-SW4(config)#snmp-server engineID local 192168183244
Lab-SW4(config)#snmp-server group GROUP1 v3 priv
Lab-SW4(config)#$User1 GROUP1 v3 auth sha AUTHPass1 priv aes 128 PRIVPass1
Lab-SW4(config)#do wri
Building configuration...
Successfully configured 192.168.183.244 & Disconnecting.
-----
-----
All tasks were completed successfully. Bye!
```

- 7 At the end of the successful SNMP configuration run, log in to each device to confirm SNMP-related configurations on each device. An example of the Lab-SW4 configuration is shown, but all other devices should have the SNMP user and group configurations.

```
Lab-SW4#show snmp user
User name: SNMPUser1
Engine ID: 192168183244
storage-type: nonvolatile          active
Authentication Protocol: SHA
Privacy Protocol: AES128
Group-name: GROUP1
Lab-SW4#show run | in snmp
snmp-server engineID local 192168183244
snmp-server group GROUP1 v3 priv
```

Your Cisco devices are now ready for SNMP communication with your SNMP server.

SNMPwalk from Linux Server

From your centos8s1 server, let's continue to perform snmpwalk using SNMPv3 commands; there is a lot to be learned here.

Task

- 1 The CentOS 8.1 server uses the following `snmpwalk` command to retrieve the OID information used by LAB-R1's vIOS MIB via SNMP. If you use this command, all OID information is loaded, as shown here:

```
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l au-
thPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.10
SNMPv2-MIB::sysDescr.0 = STRING: Cisco IOS Software,
IOSv Software (VIOS-ADVENTERPRISEK9-M), Version
15.6(2)T, RELEASE SOFTWARE (fc2)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2016 by Cisco Systems, Inc.
Compiled Tue 22-Mar-16 16:19 by prod_rel_team
```

#	Task
	<pre> SNMPv2-MIB::sysObjectID.0 = OID: SNMPv2- SMI::enterprises.9.1.1041 DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (157667) 0:26:16.67 SNMPv2-MIB::sysContact.0 = STRING: SNMPv2-MIB::sysName.0 = STRING: LAB- R1.pynetauto.local SNMPv2-MIB::sysLocation.0 = STRING: SNMPv2-MIB::sysServices.0 = INTEGER: 78 SNMPv2-MIB::sysORLastChange.0 = Timeticks: (0) 0:00:00.00 SNMPv2-MIB::sysORID.1 = OID: SNMPv2- SMI::enterprises.9.7.129 SNMPv2-MIB::sysORID.2 = OID: SNMPv2- SMI::enterprises.9.7.115 SNMPv2-MIB::sysORID.3 = OID: SNMPv2- SMI::enterprises.9.7.265 SNMPv2-MIB::sysORID.4 = OID: SNMPv2- SMI::enterprises.9.7.112 SNMPv2-MIB::sysORID.5 = OID: SNMPv2- SMI::enterprises.9.7.106 SNMPv2-MIB::sysORID.6 = OID: SNMPv2- SMI::enterprises.9.7.47 SNMPv2-MIB::sysORID.7 = OID: SNMPv2- SMI::enterprises.9.7.122 [... omitted for brevity] </pre>

If you run the same command on other devices, you should also see all MIB information for each device.

2

```

[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l au-
thPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.20
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l au-
thPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.101
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l au-
thPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.102
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l au-
thPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.133
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l au-
thPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.153
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l au-
thPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.244

```


#	Task
---	------

Practice retrieving LAB-SW1's system information using the `snmpwalk` or `snmpget` command.

1.3.6.1.2.1.1.3.0 (`sysUpTime.0`) is an OID indicating the time the system was started. You can use the `snmpget` command to retrieve and check information from the SNMP manager to the SNMP agent.

3 [pynetauto@centos8s1 snmp_test]\$ **snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X "PRIVPass1" 192.168.183.10 sysUpTime.0**

```
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks:
(1606783) 4:27:47.83
```

The SNMP agent LAB-SW1 tells us the uptime since system restart is 4 hours 27 minutes and 47 seconds.

Check a router or switch's name using 1.3.6.1.2.1.1.5.0 (`sysName.0` or `sysName`).

[pynetauto@centos8s1 snmp_test]\$ **snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X "PRIVPass1" 192.168.183.10 sysName.0**

```
SNMPv2-MIB::sysName.0 = STRING: LAB-
R1.pynetauto.local
```

4 [pynetauto@centos8s1 snmp_test]\$ **snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X "PRIVPass1" 192.168.183.10 sysName**

```
SNMPv2-MIB::sysName.0 = STRING: LAB-
R1.pynetauto.local
```

[pynetauto@centos8s1 snmp_test]\$ **snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X "PRIVPass1" 192.168.183.10 1.3.6.1.2.1.1.5.0**

```
SNMPv2-MIB::sysName.0 = STRING: LAB-
R1.pynetauto.local
```

5 If you use 1.3.6.1.2.1.2.2.1.7(`ifAdminStatus.1`), you can check the interface's setting status. This information confirms that the administrator has activated the interface.

[pynetauto@centos8s1 snmp_test]\$ **snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X "PRIVPass1" 192.168.183.10 ifAdminStatus**

```
IF-MIB::ifAdminStatus.1 = INTEGER: up(1)
IF-MIB::ifAdminStatus.2 = INTEGER: up(1)
IF-MIB::ifAdminStatus.3 = INTEGER: down(2)
IF-MIB::ifAdminStatus.4 = INTEGER: down(2)
IF-MIB::ifAdminStatus.5 = INTEGER: up(1)
IF-MIB::ifAdminStatus.6 = INTEGER: up(1)
IF-MIB::ifAdminStatus.7 = INTEGER: up(1)
```

[pynetauto@centos8s1 snmp_test]\$ **snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X**

#	Task
---	------

```
"PRIVPass1" 192.168.183.10 1.3.6.1.2.1.2.2.1.7

IF-MIB::ifAdminStatus.1 = INTEGER: up(1)
IF-MIB::ifAdminStatus.2 = INTEGER: up(1)
IF-MIB::ifAdminStatus.3 = INTEGER: down(2)
IF-MIB::ifAdminStatus.4 = INTEGER: down(2)
IF-MIB::ifAdminStatus.5 = INTEGER: up(1)
IF-MIB::ifAdminStatus.6 = INTEGER: up(1)
IF-MIB::ifAdminStatus.7 = INTEGER: up(1)
```

6

As you may have noticed previously, the OID of the first interface is the ID with .1 appended after 1.3.6.1.2.1.2.2.1.7. If .1 is added and executed, as shown next, the status of the GigabitEthernet0/0 port is displayed as up(1).

```
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.10 1.3.6.1.2.1.2.2.1.7.1
IF-MIB::ifAdminStatus.1 = INTEGER: up(1)
```

7

Likewise, adding .3 at the end means GigabitEthernet0/1 . This port is currently disabled, so it is displayed as down(2).

```
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.10 1.3.6.1.2.1.2.2.1.7.3
IF-MIB::ifAdminStatus.3 = INTEGER: down(2)
```

8

If you use OID 1.3.6.1.2.1.2.2.1.8 (ifOperStatus), you can query all the interfaces' operational status . Quickly run a snmpwalk command from your server for confirmation.

```
[pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.10 ifOperStatus
IF-MIB::ifOperStatus.1 = INTEGER: up(1)
IF-MIB::ifOperStatus.2 = INTEGER: up(1)
IF-MIB::ifOperStatus.3 = INTEGER: down(2)
IF-MIB::ifOperStatus.4 = INTEGER: down(2)
IF-MIB::ifOperStatus.5 = INTEGER: up(1)
IF-MIB::ifOperStatus.6 = INTEGER: up(1)
IF-MIB::ifOperStatus.7 = INTEGER: up(1)
```

9

Use the snmpget command to query information about an interface, in this case, GigabitEthernet0/0. IfDescr.1 queries the interface's description, and ifOperStatus.1 queries the interface's operational status. You can replace the number 1 with 2 to query the next GigabitEthernet2 interface.

```
[pynetauto@centos8s1 snmp_test]$ snmpget -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X
"PRIVPass1" 192.168.183.10 ifDescr.1 ifOperStatus.1
IF-MIB::ifDescr.1 = STRING: GigabitEthernet0/0
```

#	Task
---	------

	IF-MIB::ifOperStatus.1 = INTEGER: up(1)
--	---

The following `snmpget` command queries for the interface status of GigabitEthernet0/3 ; since it is not in use, it is marked as down (2).

```

10 [pynetauto@centos8s1 snmp_test]$ snmpget -v3 -l auth-Priv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X "PRIVPass1" 192.168.183.10 ifDescr.4 ifOperStatus.4
IF-MIB::ifDescr.4 = STRING: GigabitEthernet0/3
IF-MIB::ifOperStatus.4 = INTEGER: down(2)

```

Many OIDs can reveal each component and configuration's current status. Another fact worth noting is the RMON OID; it can reveal the software version number and the booting information. The example shows the router RMON OID.

```

11 [pynetauto@centos8s1 snmp_test]$ snmpwalk -v3 -l authPriv -u SNMPUser1 -a SHA -A "AUTHPass1" -x AES -X "PRIVPass1" 192.168.183.10 rmon
RMON-MIB::rmon.19.1.0 = Hex-STRING: FF C0 00 40
RMON-MIB::rmon.19.2.0 = STRING: "15.6(2)T"
RMON-MIB::rmon.19.3.0 = ""
RMON-MIB::rmon.19.4.0 = Hex-STRING: 07 E4 08 1E 02 15 16 07
RMON-MIB::rmon.19.5.0 = INTEGER: 1
RMON-MIB::rmon.19.6.0 = STRING: "flash0:/vios-adventerprisek9-m"
RMON-MIB::rmon.19.7.0 = IpAddress: 0.0.0.0
RMON-MIB::rmon.19.8.0 = INTEGER: 1
RMON-MIB::rmon.19.9.0 = INTEGER: 1
RMON-MIB::rmon.19.12.0 = IpAddress: 0.0.0.0
RMON-MIB::rmon.19.15.0 = Hex-STRING: 00
RMON-MIB::rmon.19.16.0 = Hex-STRING: 7E 00

```

The OID information used in vIOS is almost identical to real IOS equipment. There is a lot you can study from this lab and gain useful information from the previous examples. Once again, you must type the commands yourself on the keyboard and learn the previous concept. See Figure [15-11](#).



For Cisco networking devices, you can find the required OID information using the Cisco Feature Navigator .

URL: <https://cfnng.cisco.com/mibs>

For some Q&A on Cisco MIBs, refer to the following:

URL: <https://www.cisco.com/c/en/us/support/docs/ip/simple-network-management-protocol-snmp/9226-mibs-9226.html>

For your information, the following MIBs are among the most useful for Cisco devices:

- IF-MIB: Interface counter

URL: <https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/interface/configuration/15-s/ir-15-s-book/ir-if-mibs.html>

- IP-MIB: Contains IP address

- IP-FORWARD-MIB: Contains a routing table

- ENTITY-MIB: Contains inventory information

- LLDP-MIB: Contains neighbor information

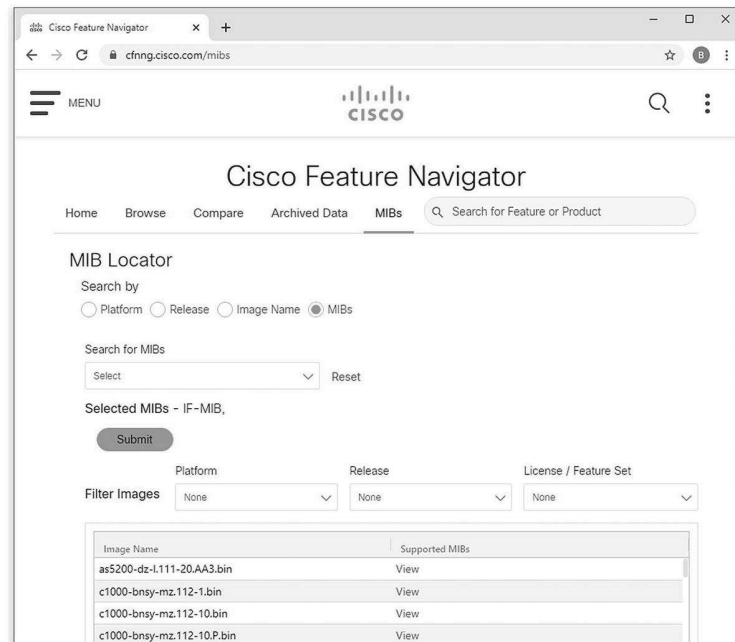


Figure 15-11. Cisco Feature Navigator, MIBs

Borrowing Some Example Python SNMP Code

As discussed throughout this book, learning a single programming language such as Python, JavaScript, or Perl does not turn you into a network engineer who is immediately ready to develop applications. Based on your strong foundational network knowledge, you have to try your best to broaden your supported technologies. Your journey might become a long haul, perhaps painfully too slow for some people. Luckily, you can often find the right code on the Internet for your use and borrow it. Why bother reinventing the wheel? If you are not the first person to come across a specific problem, then somebody else has solved the problem ahead of you. You just have to know how and where to look for the specific information you are looking for. (Of course, I believe that some knowledge is more dependable and better transferred in other forms.)

On the web, I was looking for some Python use cases with SNMP and came across a good article and bookmarked the site for my future reference. When the time came to write this book, I was asked to include SNMP and Python as part of this book. But I did not want to spend weeks trying to write new code; I wanted to find an existing SNMP Python script to show an example of how Python is used in SNMP. The SNMP Python code that will be introduced in this section is sourced from Alessandro Maggio's home page. Thanks to Alessandro, I have included the script as part of this book with his consent. To get the full explanation of the code, please visit Alex's blog .



Find Alessandro Maggio's blog and code tutorial here:

URL: <https://www.ictshore.com/>

URL: <https://www.ictshore.com/sdn/python-snmp-tutorial/>

SNMP Lab Source Code

The following code is the Python code for SNMPv2c. SNMPv2c uses read-only community strings, simplifying SNMP configuration on both the server and agent sides. Still, as we all know, SNMPv2c is less secure than SNMPv3, so you will quickly review the code and make minor modifications to convert it into SNMPv3 code. Later, based on a scenario, we will write a simple Python integration script to work in harmony with this script. First, let's take a closer look at the original SNMP Python code. The explanations always begin with # or """ """.

```
[pynetauto@centos8s1 snmp_test]$ nano quicksnmp.py
GNU nano 4.8 /home/pynetauto/snmp_test/quicksnmp.py
# This code uses the High-Level API module included in the pysnmp library.
from pysnmp import hlapi
"""
The first function or get () function defines pre-requisite information to initiate and make device inf
"""
def get(target, oids, credentials, port=161, engine=hlapi.SnmpEngine(), context=hlapi.ContextData()):
    handler = hlapi.getCmd(
        engine,
        credentials,
        hlapi.UdpTransportTarget((target, port)),
        context,
        *construct_object_types(oids)
    )
    return fetch(handler, 1)[0]
# Second function needs a single argument list_of_oids, a blank list is used to collect and return obje
# information.
def construct_object_types(list_of_oids):
    object_types = []
    for oid in list_of_oids:
        object_types.append(hlapi.ObjectType(hlapi.ObjectIdentity(oid)))
    return object_types
# Third function requires two arguments and uses the result list to handle any errors in the script
.
def fetch(handler, count):
    result = []
    for i in range(count):
```

```

    try:
        error_indication, error_status, error_index, var_binds = next(handler)
        if not error_indication and not error_status:
            items = {}
            for var_bind in var_binds:
                items[str(var_bind[0])] = cast(var_bind[1])
            result.append(items)
        else:
            raise RuntimeError('Got SNMP error: {}'.format(error_indication))
    except StopIteration:
        break
    return result

# The fourth function, cast() uses the information received from PySNMP to pass the values, this functi
# plays the role of changing the value into int or float or string type.
def cast(value):
    try:
        return int(value)
    except (ValueError, TypeError):
        try:
            return float(value)
        except (ValueError, TypeError):
            try:
                return str(value)
            except (ValueError, TypeError):
                pass
    return value

# This code is written for SNMPv2c and uses a community string named 'ICTSHORE'.
hlapi.CommunityData('ICTSHORE')
# If SNMPv3 is used, a user ID and two authentication keys should be used. The script (or the Server) u
# this information to communicate with the desired SNMP Agent to get device information.
hlapi.UsmUserData('testuser', authKey='authenticationkey', privKey='encryptionkey',
authProtocol=hlapi.usmHMACSHAAuthProtocol, privProtocol=hlapi.usmAesCfb128Protocol)
# Specifies the specific OID to obtain specified information from the SNMP Agent. '1.3.6.1.2.1.1.5.0' d

.
print(get('192.168.47.10', ['1.3.6.1.2.1.1.5.0'], hlapi.CommunityData('ICTSHORE')))
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line

```

Source code: <https://www.ictshore.com/sdn/python-snmp-tutorial/>

SNMPv3 Python Code to Query Cisco Device Information

Now you are going to reiterate the previous SNMPv2c Python code to support SNMPv3. By studying someone else's code and testing the script in a lab environment, you can gain more insight into how Python interacts with different applications and behaves. This is also a perfect opportunity to watch and learn the real difference between SNMPv2c and SNMPv3 in action.

Task

For the server (centos8s1) to communicate with network devices over SNMP, you will first install the server's pynsnmp library . Complete the installation using the `sudo pip3 install pynsnmp` command.

```
[pynetauto@centos8s1 ~]$ sudo pip3 install pynsnmp
```

- After creating an SNMP Python code called `quicksnmp_v3.py`, open the file with the nano editor and copy and paste all lines of code from `quicksnmp.py`

Task

```

into quicksnmp_v3.py.
[pynetauto@centos8s1 ~]$ cd snmp_test
[pynetauto@centos8s1 snmp_test]$ nano quicksnmp_v3.py

```

3 The following modifications are required to turn quicksnmp.py into the SNMP version 3 script.

First, when you read the documentation on [pysnmplabs.com](http://snmplabs.com/pysnmp), you will see that you need to import and use `from pysnmp.hlapi import UsmUserData`.

<http://snmplabs.com/pysnmp/docs/api-reference.html#pysnmp.hlapi.UsmUserData>

A. Add `from pysnmp.hlapi import UsmUserData` at the top of your code, as shown here:

```

GNU nano
4.8                               /home/pynetauto/snmp_test/quicksnmp_v3.py
from pysnmp import hlapi
from pysnmp.hlapi import UsmUserData
def get(target, oids, credentials, port=161,
engine=hlapi.SnmpEngine(), context=hlapi.ContextData()):
    handler = hlapi.getCmd(
        engine,
        credentials,
        hlapi.UdpTransportTarget((target, port)),
        context,
        *construct_object_types(oids)
    )
    return fetch(handler, 1)[0]
def construct_object_types(list_of_oids):
    object_types = []
    for oid in list_of_oids:
        object_types.append(hlapi.ObjectType(hlapi.ObjectIdentity(oid)))
    return object_types
def fetch(handler, count):
    result = []
    for i in range(count):
        try:
            error_indication, error_status, error_index, var_binds =
next(handler)
            if not error_indication and not error_status:
                items = {}
                for var_bind in var_binds:
                    items[str(var_bind[0])] = cast(var_bind[1])
                result.append(items)
            else:
                raise RuntimeError('Got SNMP error:
{0}'.format(error_indication))
        except StopIteration:
            break
    return result

```

Task

```
def cast(value):
    try:
        return int(value)
    except (ValueError, TypeError):
        try:
            return float(value)
        except (ValueError, TypeError):
            try:
                return str(value)
            except (ValueError, TypeError):
                pass
    return value

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J
Justify      ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace  ^U Paste Text  ^T To
Spell  ^_ Go To Line
```

B. Next, you need to enter your user information, authentication key, and privacy key. As shown in the earlier SNMP user configuration, the SNMPv3 user, `SNMPUser1`, was configured with SHA authentication keys and the AES128 privacy key. So, add the following line exactly as shown here:

```
hlapi.UsmUserData('SNMPUser1', authKey="AUTHPass1",
privKey="PRIVPass1", authProtocol=hlapi.usmHMACSHAAuthProtocol,
privProtocol=hlapi.usmAesCfb128Protocol)
```

C. Enter the `print` statement so that the `get()` function can pass the correct agent IP information and other credential variables to print the information on the screen.

```
print(get('192.168.183.10', ['1.3.6.1.2.1.1.5.0'],
hlapi.UsmUserData('SNMPUser1', authKey="AUTHPass1",
privKey="PRIVPass1", authProtocol=hlapi.usmHMACSHAAuthProtocol,
privProtocol=hlapi.usmAesCfb128Protocol)))
```

4

Use dot OID `'1.3.6.1.2.1.1.5.0'` in the last line of code and execute the previous Python code to retrieve and print the device name of 192.168.183.10.

```
[pynetauto@centos8s1 snmp_test]$ python3 quicksnmp_v3.py
{'1.3.6.1.2.1.1.5.0': 'LAB-R1.pynetauto.local'}
```


Task

If you want to retrieve the operational status of GigabitEthernet0/0 , you can add another dot OID line with an OID value of '1.3.6.1.2.1.2.2.1.7.1' in the last line. When you run the changed script, the interface state will be returned as a number: 1 for On and 2 for Off.

D. Add the following line to the end of `quicksnmp_v3.py`:

```
5 print(get('192.168.183.10', ['1.3.6.1.2.1.2.2.1.7.1'],
    hlapi.UsmUserData('SNMPUser1', authKey="AUTHPass1",
    privKey='PRIVP$
52
[pynetauto@centos8s1 snmp_test]$ python3 quicksnmp_v3.py
{'1.3.6.1.2.1.1.5.0': 'LAB-R1.pynetauto.local'}
{'1.3.6.1.2.1.2.2.1.7.1': 1}
```

When you increase the last digit and run the script for the GigabitEthernet 0/3 interface status, it returns 2. It is not connected to anything. 4 is the interface reference number, and 2 denotes the Off status.

```
6 [pynetauto@centos8s1 snmp_test]$ python3 quicksnmp_v3.py
{'1.3.6.1.2.1.1.5.0': 'LAB-R1.pynetauto.local'}
{'1.3.6.1.2.1.2.2.1.7.4': 2}
```

Using Python SNMP Code to Run a Query for an Interface Description

This lab is a continuation of the previous one, and you first have to make a copy of the `quicksnmp_v3.py` script and create another script to complete it. The following script will let you run SNMP queries on interfaces and their descriptions using the SNMPv3 Python code. Since this book is an introductory book that tries to introduce as much Python and network automation as possible, we will not go too deep into SNMP in-depth knowledge, leaving that task for the Cisco documentation. What is important here is to learn how Python is used to interact with Cisco devices using basic SNMP MIB and OID information.

Task

Now, using the Linux `cp` command , copy `quicksnmp_v3.py` and create another file named `quicksnmp_v3_get_bulk_auto.py`.

```
1 [pynetauto@centos8s1 snmp_test]$ cp quicksnmp_v3.py
    quicksnmp_v3_get_bulk_auto.py
[pynetauto@centos8s1 snmp_test]$ nano
    quicksnmp_v3_get_bulk_auto.py
```

2 At the end of the line, add the following code, and run it. You are appending the next line of code at the bottom. Also, make sure you use the SW3 IP address of 192.168.183.153. What this code will do is run the SNMP query on the interface's name and descriptions.

```
GNU nano
4.8      /home/pynetauto/snmp_test/quicksnmp_v3_get_bulk_auto.py
[... omitted for brevity]
def cast(value):
    try:
```

```
# Task

    return int(value)

except (ValueError, TypeError):

    try:

        return float(value)

    except (ValueError, TypeError):

        try:

            return str(value)

        except (ValueError, TypeError):

            pass

    return value # existing quicksnmp_v3.py

#Append the following to the end of the code

def get_bulk(target, oids, credentials, count, start_from=0,
port=161, engine=hlapi.SnmpEngine(),
context=hlapi.ContextData()):

    handler = hlapi.bulkCmd(

        engine,

        credentials,

        hlapi.UdpTransportTarget((target, port)),

        context,

        start_from, count,

        *construct_object_types(oids)

    )

    return fetch(handler, count)

def get_bulk_auto(target, oids, credentials, count_oid,
start_from=0, port=161, engine=hlapi.SnmpEngine(),
context=hlapi.ContextData()):

    count = get(target, [count_oid], credentials, port, engine,
context)[count_oid]

    return get_bulk(target, oids, credentials, count, start_from,
port, engine, context)

its = get_bulk_auto('192.168.183.153', ['1.3.6.1.2.1.2.2.1.2 ',
'1.3.6.1.2.1.31.1.1.1.18'], hlapi.UsmUserData('SNMPUser1',
authKey="AUTHPass1", privKey="PRIVPass1",
authProtocol=hlapi.usmHMACSHAAuthProtocol,
privProtocol=hlapi.usmAesCfb128Protocol), '1.3.6.1.2.1.2.1.0')

for it in its:

    for k, v in it.items():

        print("{0}={1}".format(k, v))

    print('')

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J
Justify      ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To
Spell  ^_ Go To Line
```

- 3 Finally, run the `quicksnmp_v3_get_bulk_auto.py` code, and you should get the interface names and their descriptions; since we only configured

Task

the description for `Gi0/0` and `vlan1`, you will see the names of these interfaces.

```
[pynetauto@centos8s1 snmp_test]$ python3
quicksnmp_v3_get_bulk_auto.py
{'1.3.6.1.2.1.1.5.0': 'Lab-sw3.pynetauto.com'}
1.3.6.1.2.1.2.2.1.2.1=GigabitEthernet0/0
1.3.6.1.2.1.31.1.1.1.18.1="1GB Main Connection"
1.3.6.1.2.1.2.2.1.2.2=GigabitEthernet0/1
1.3.6.1.2.1.31.1.1.1.18.2=
[... omitted for brevity]
1.3.6.1.2.1.2.2.1.2.16=GigabitEthernet3/3
1.3.6.1.2.1.31.1.1.1.18.16=
1.3.6.1.2.1.2.2.1.2.17=NULL0
1.3.6.1.2.1.31.1.1.1.18.17=
1.3.6.1.2.1.2.2.1.2.18=Vlan1
1.3.6.1.2.1.31.1.1.1.18.18="Native interface"
```

There are hundreds of SNMP OID objects you can query and learn about. If you have a little bit of spare time, you can explore more about Cisco Device MIBs, especially the CPU and processes related to MIB information. Here is a Cisco link that shows you how to query CPUs using SNMP:

URL: <https://www.cisco.com/c/en/us/support/docs/ip/simple-network-management-protocol-snmp/15215-collect-cpu-util-snmp.html>

Summary

In this chapter, you learned how to use the Linux scheduler `cron` on Ubuntu and CentOS. You can now schedule your Python applications to run at a specified time or at specified intervals. Once you finish developing an application, you can use `cron` to run the tasks even at odd hours of the day, and you will not lose any beauty sleep. The tool `cron` is a powerful, free, and open source task scheduler, and whenever we can, we should rely on open source tools. You also learned the SNMP foundations and looked at the `SNMPwalk` examples using the `pysnmp` module; in the process, we borrowed another developer's work so that we did not have to reinvent the wheel. This can save you a lot of time.

With SNMP and Python, there is a lot we can do, but that would be another entire book. In Chapter 16, you will be exploring even more network automation scenarios. Also, you will learn to use `virtualenv`, the virtual environment used for Python testing as well as Ansible, pyATS and Docker, sendmail, and Twilio SMS Python network automation application development.